



**FACULTAD DE SISTEMAS Y
TELECOMUNICACIONES**

CARRERA DE SOFTWARE



FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

INGENIERÍA EN SOFTWARE

MATERIA:

PROGRAMACIÓN FUNCIONAL

DOCENTE:

Ing. ANTHONY ABRAHAN PACHAY ESPINOZA

NOMBRE:

DAYWIN JOAN SALVATIERRA COMANICHO

SEMESTRE:

QUINTO SEMESTRE 5/1

PERÍODO:

2026-2

Taller de programación funcional en Python

Funciones de orden superior, closures y lambdas

El taller reúne veinte ejercicios en cuatro niveles. Cada página presenta una explicación breve, una captura del código abierto en Visual Studio Code con tema oscuro y la salida obtenida al ejecutar ese archivo en Python.

Repositorio público

<https://github.com/DaywinJC/taller-programacion-funcional>

Nivel 1: transformaciones y reglas recibidas como funciones. Nivel 2: contadores, acumuladores y estados.

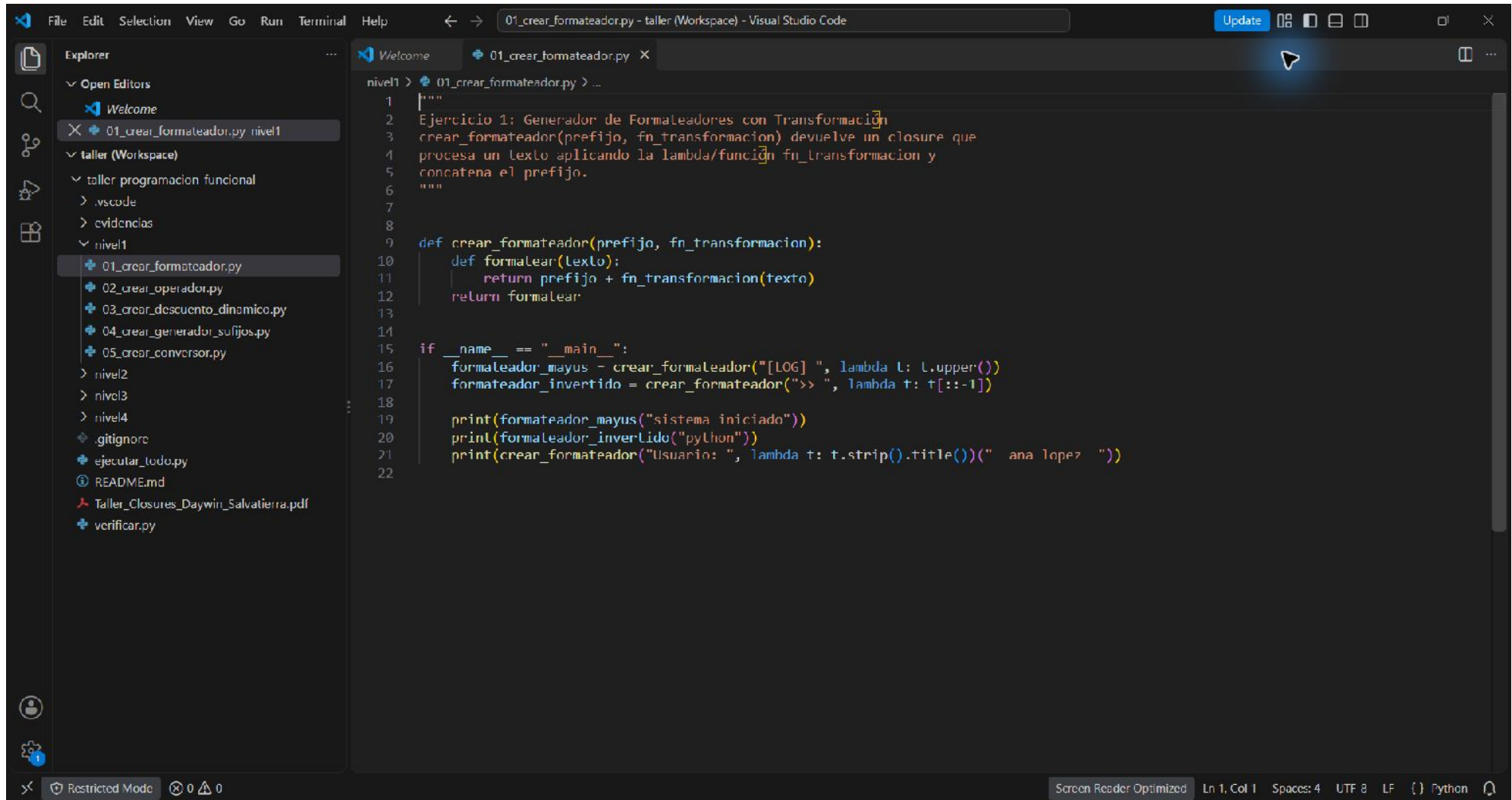
Nivel 3: colecciones, composición e historial. Nivel 4: validación, caché, pipelines, eventos y consultas.

Para ejecutar todos los ejemplos: `python ejecutar_todo.py`. Para comprobar salidas y casos límite: `python verificar.py`. El proyecto requiere Python 3.10 o posterior y no utiliza paquetes externos.

Cada uno de los veinte archivos finalizó sin errores. Los valores aleatorios usan una semilla fija; el tiempo de auditoría puede variar.

1. Formateadores con transformación

El closure conserva el prefijo y la función que transforma el texto. Así se pueden crear varios formateadores sin repetir la misma lógica. En las pruebas, uno convierte a mayúsculas, otro invierte el texto y el último limpia los espacios del nombre.



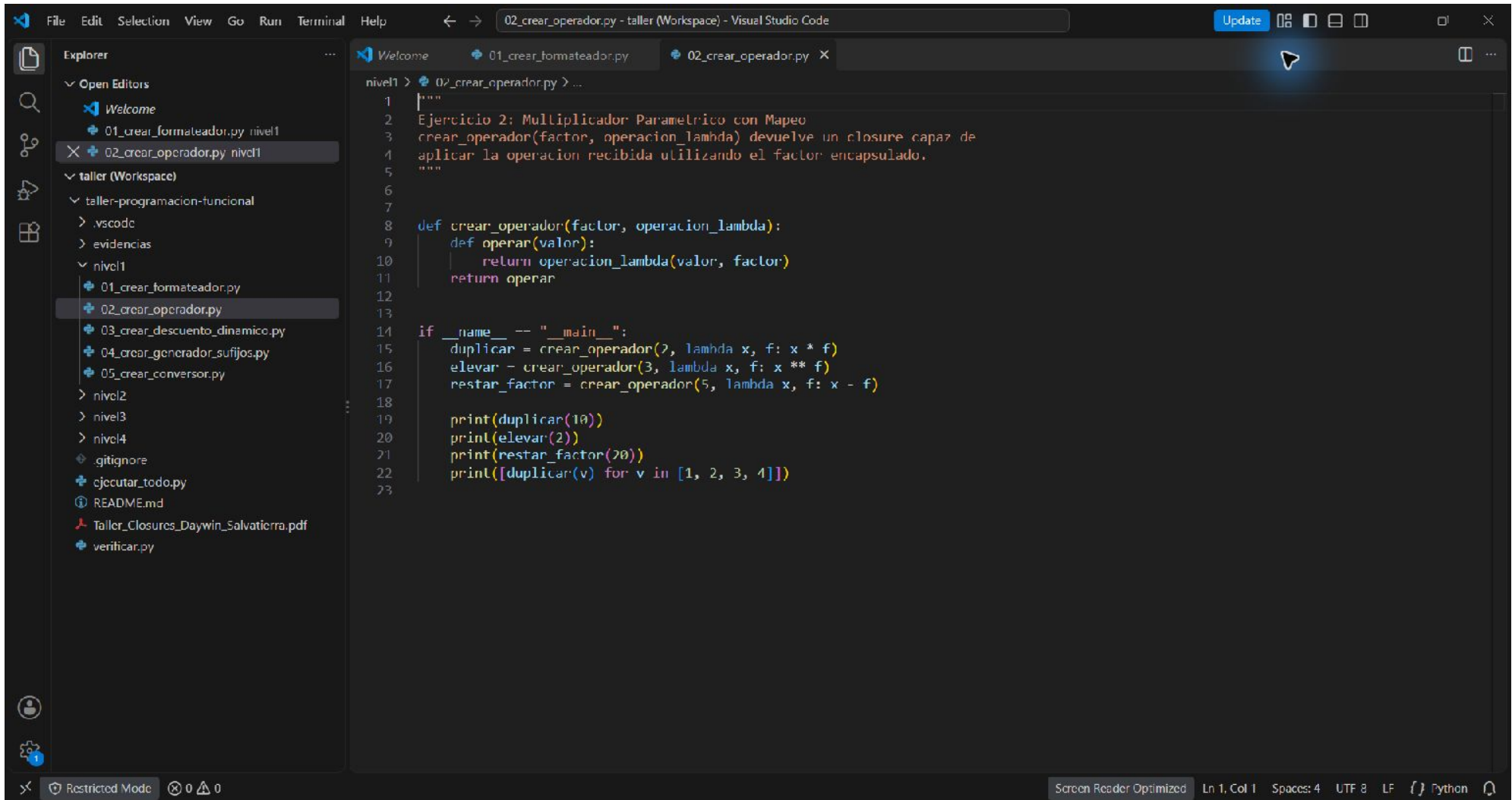
```
1 """
2 Ejercicio 1: Generador de Formateadores con Transformación
3 crear_formateador(prefijo, fn_transformacion) devuelve un closure que
4 procesa un texto aplicando la lambda/función fn_transformacion y
5 concatena el prefijo.
6 """
7
8
9 def crear_formateador(prefijo, fn_transformacion):
10     def formatear(texto):
11         return prefijo + fn_transformacion(texto)
12     return formatear
13
14
15 if __name__ == "__main__":
16     formateador_mayus = crear_formateador("[LOG] ", lambda t: t.upper())
17     formateador_invertido = crear_formateador(">> ", lambda t: t[::-1])
18
19     print(formateador_mayus("sistema iniciado"))
20     print(formateador_invertido("python"))
21     print(crear_formateador("Usuario: ", lambda t: t.strip().title())(" ana lopez "))
22
```

Salida:

```
[LOG] SISTEMA INICIADO
>> nohtyp
Usuario: Ana Lopez
```

2. Operador con un factor fijo

La función devuelta recibe un valor y lo combina con el factor capturado. La lambda decide si se multiplica, se calcula una potencia o se resta. El mismo operador también se puede aplicar a todos los elementos de una lista.



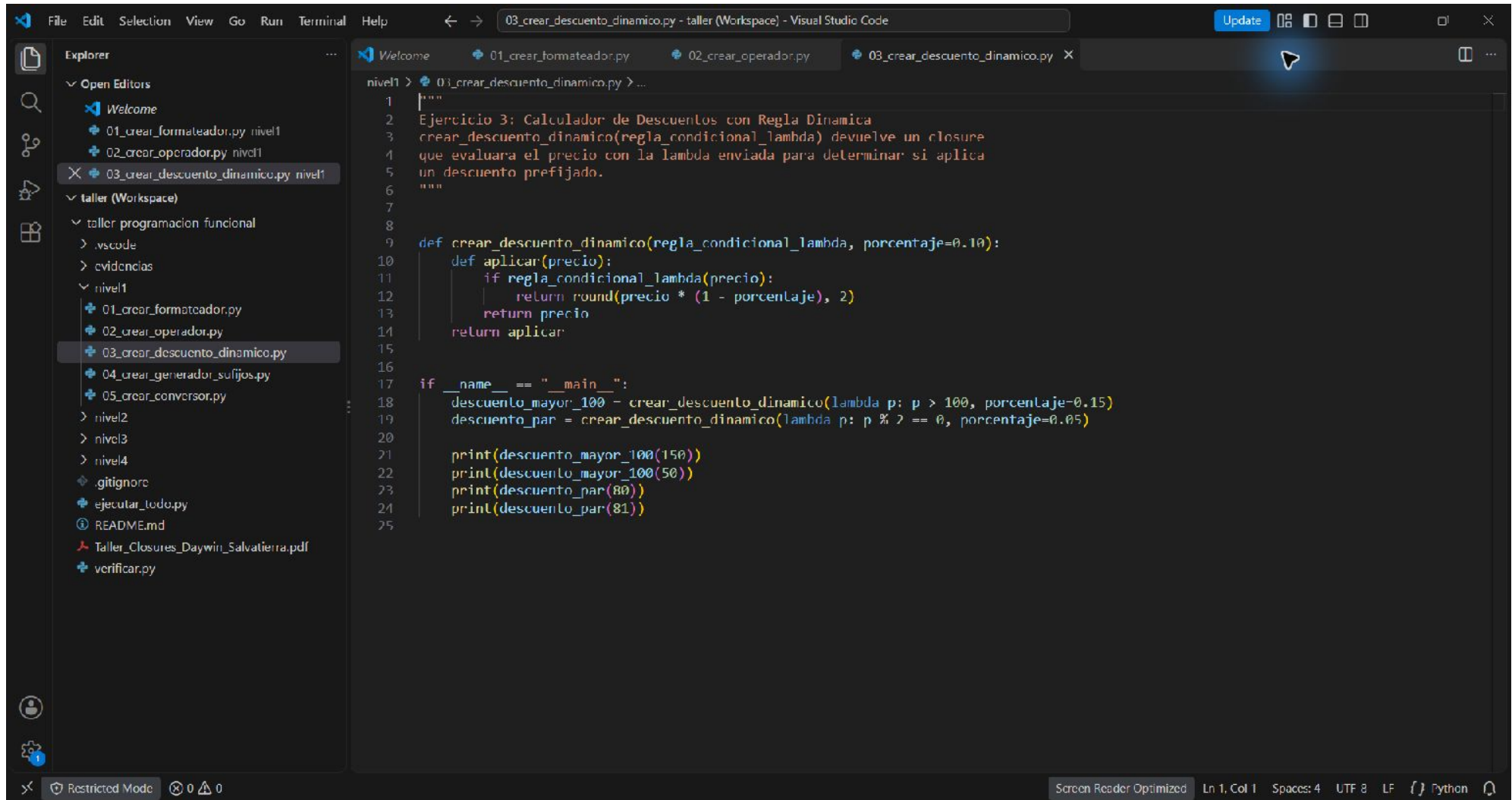
```
1 """
2 Ejercicio 2: Multiplicador Parametrico con Mapeo
3 crear_operador(factor, operacion_lambda) devuelve un closure capaz de
4 aplicar la operacion recibida utilizando el factor encapsulado.
5 """
6
7
8 def crear_operador(factor, operacion_lambda):
9     def operar(valor):
10         return operacion_lambda(valor, factor)
11     return operar
12
13
14 if __name__ == "__main__":
15     duplicar = crear_operador(2, lambda x, f: x * f)
16     elevar = crear_operador(3, lambda x, f: x ** f)
17     restar_factor = crear_operador(5, lambda x, f: x - f)
18
19     print(duplicar(10))
20     print(elevar(2))
21     print(restar_factor(20))
22     print([duplicar(v) for v in [1, 2, 3, 4]])
23
```

Salida:

```
20
8
15
[2, 4, 6, 8]
```

3. Descuento con una regla dinámica

La condición determina qué precios reciben el descuento. Para precios mayores de 100 se aplica un 15 %; en el otro ejemplo se descuenta un 5 % a los precios pares. Los valores que no cumplen la regla se devuelven sin cambios.



The screenshot shows the Visual Studio Code interface with a workspace named 'taller (Workspace)'. The Explorer panel on the left shows the file structure, including a folder 'nivel1' containing several Python files. The file '03_crear_descuento_dinamico.py' is selected and open in the editor. The code in the editor is as follows:

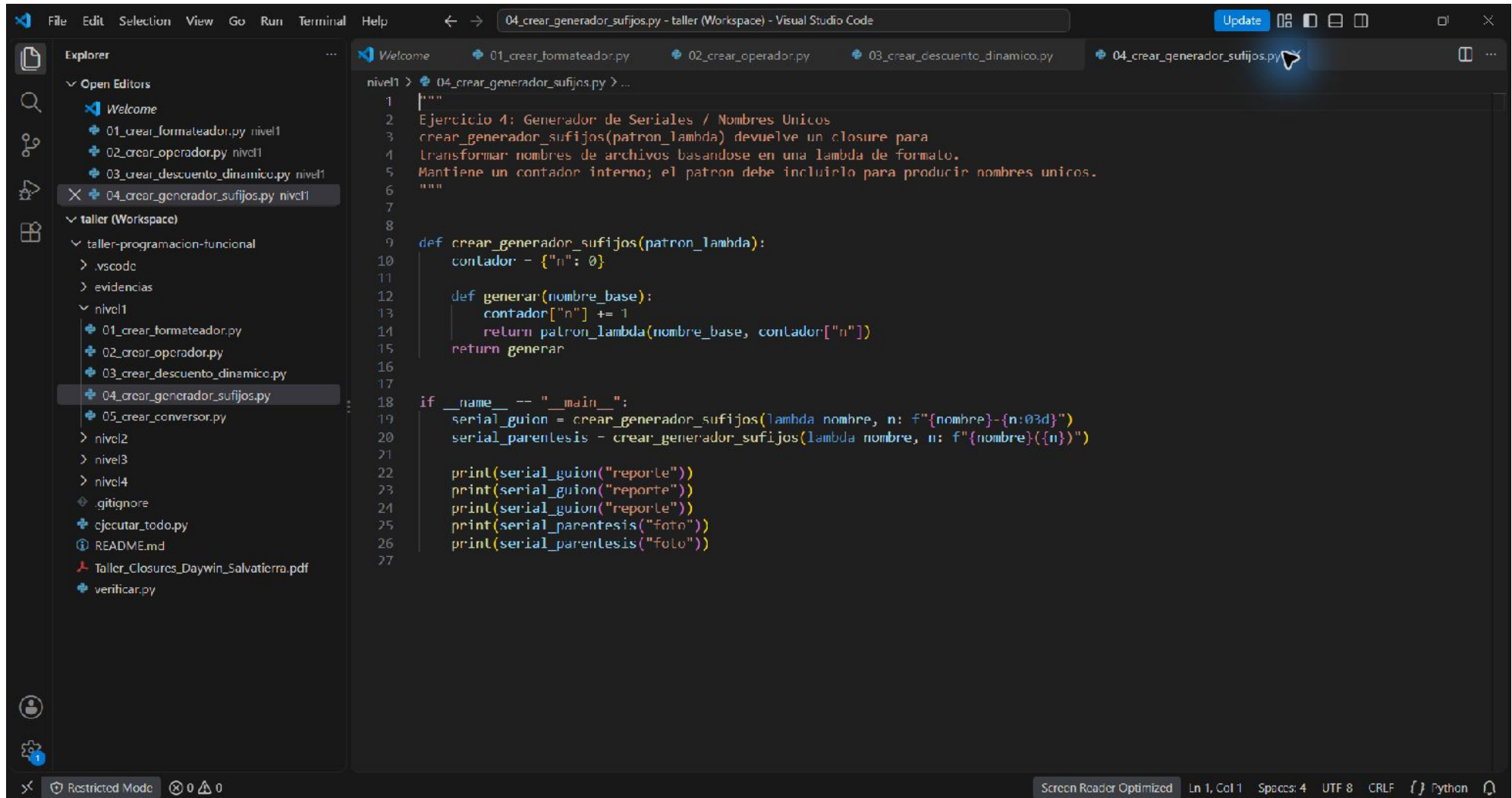
```
1 """
2 Ejercicio 3: Calculador de Descuentos con Regla Dinamica
3 crear_descuento_dinamico(regla_condicional_lambda) devuelve un closure
4 que evaluara el precio con la lambda enviada para determinar si aplica
5 un descuento prefijado.
6 """
7
8
9 def crear_descuento_dinamico(regla_condicional_lambda, porcentaje=0.10):
10     def aplicar(precio):
11         if regla_condicional_lambda(precio):
12             return round(precio * (1 - porcentaje), 2)
13         return precio
14     return aplicar
15
16
17 if __name__ == "__main__":
18     descuento_mayor_100 = crear_descuento_dinamico(lambda p: p > 100, porcentaje=0.15)
19     descuento_par = crear_descuento_dinamico(lambda p: p % 2 == 0, porcentaje=0.05)
20
21     print(descuento_mayor_100(150))
22     print(descuento_mayor_100(50))
23     print(descuento_par(80))
24     print(descuento_par(81))
25
```

Salida:

```
127.5
50
76.0
81
```

4. Generador de nombres con sufijos

Cada llamada incrementa un contador privado y lo entrega a la lambda junto con el nombre base. El patrón decide cómo presentar el número. Los dos generadores de la prueba mantienen contadores independientes.



The screenshot shows the Visual Studio Code editor with a workspace named 'taller (Workspace)'. The Explorer sidebar on the left shows a file tree with 'nivel1' containing several Python files, including '04_crear_generador_sufijos.py' which is currently selected. The main editor window displays the code for '04_crear_generador_sufijos.py'. The code defines a function 'crear_generador_sufijos' that takes a 'patron_lambda' and returns a closure 'generar'. The 'generar' function increments a private counter and calls 'patron_lambda' with the base name and the counter. A main block at the bottom demonstrates the usage, creating two generators and printing their outputs.

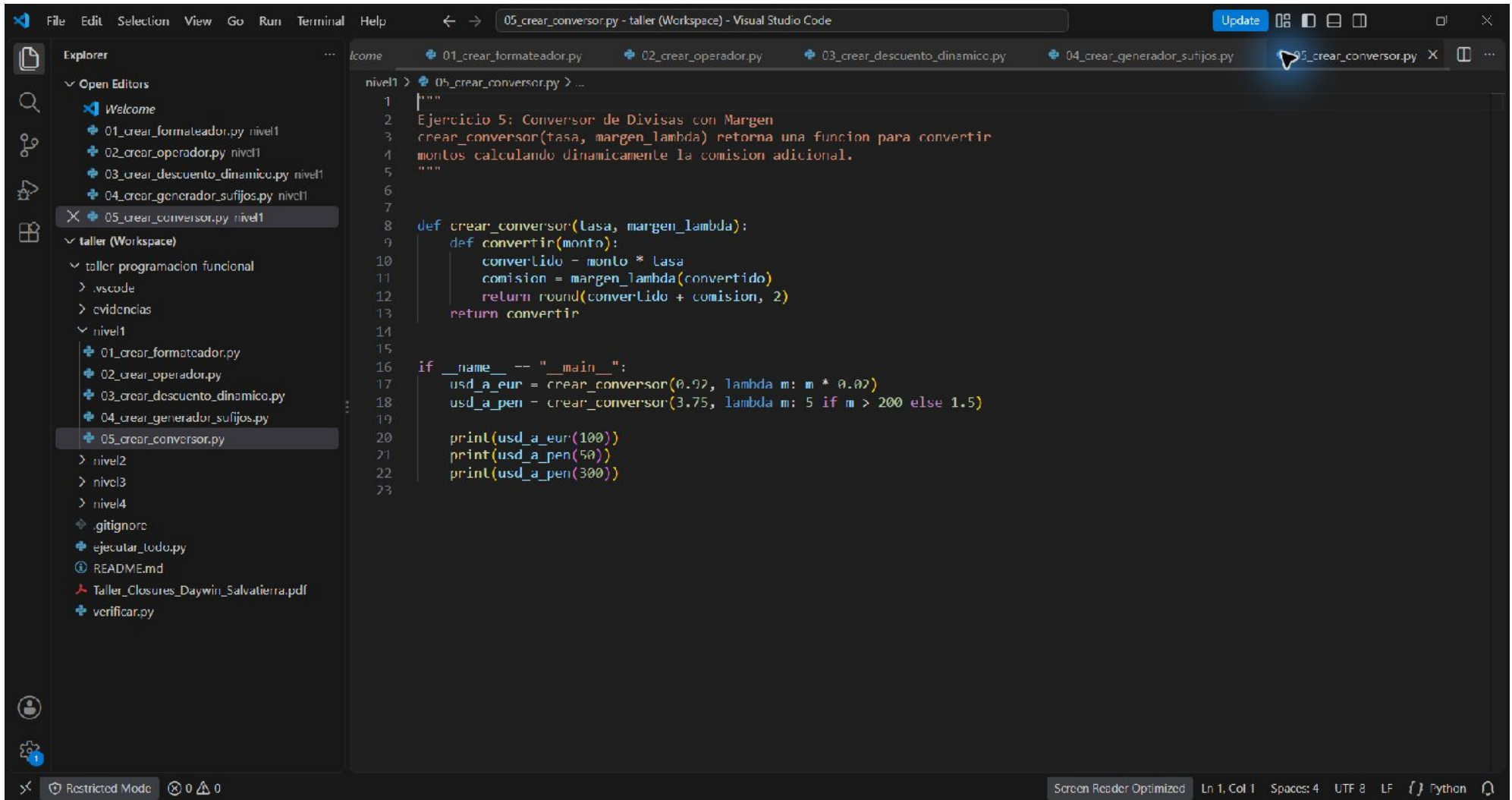
```
1 """
2 Ejercicio 4: Generador de Seriales / Nombres Unicos
3 crear_generador_sufijos(patron_lambda) devuelve un closure para
4 transformar nombres de archivos basandose en una lambda de formato.
5 Mantiene un contador interno; el patron debe incluirlo para producir nombres unicos.
6 """
7
8
9 def crear_generador_sufijos(patron_lambda):
10     contador = {"n": 0}
11
12     def generar(nombre_base):
13         contador["n"] += 1
14         return patron_lambda(nombre_base, contador["n"])
15     return generar
16
17
18 if __name__ == "__main__":
19     serial_guion = crear_generador_sufijos(lambda nombre, n: f"{nombre}-{n:03d}")
20     serial_parentesis = crear_generador_sufijos(lambda nombre, n: f"{nombre}({n})")
21
22     print(serial_guion("reporte"))
23     print(serial_guion("reporte"))
24     print(serial_guion("reporte"))
25     print(serial_parentesis("foto"))
26     print(serial_parentesis("foto"))
27
```

Salida:

```
reporte-001
reporte-002
reporte-003
foto(1)
foto(2)
```

5. Conversión de divisas con comisión

Primero se multiplica el monto por la tasa y luego se calcula el recargo con la lambda. La primera regla cobra un 2 %; la segunda usa una comisión fija que depende del importe convertido. Las tasas son datos de ejemplo.



The screenshot shows the Visual Studio Code interface with a workspace named 'taller (Workspace)'. The Explorer sidebar on the left shows a project structure with folders 'taller programacion funcional', '.vscode', 'evidencias', and 'nivel1'. The 'nivel1' folder contains five Python files: '01_crear_formateador.py', '02_crear_operador.py', '03_crear_descuento_dinamico.py', '04_crear_generador_sufijos.py', and '05_crear_convertor.py'. The '05_crear_convertor.py' file is selected and its code is visible in the editor. The code defines a function 'crear_convertor' that takes a rate and a lambda function for commission, and a main block that uses it to calculate conversions for EUR and PEN with different rates and commission rules.

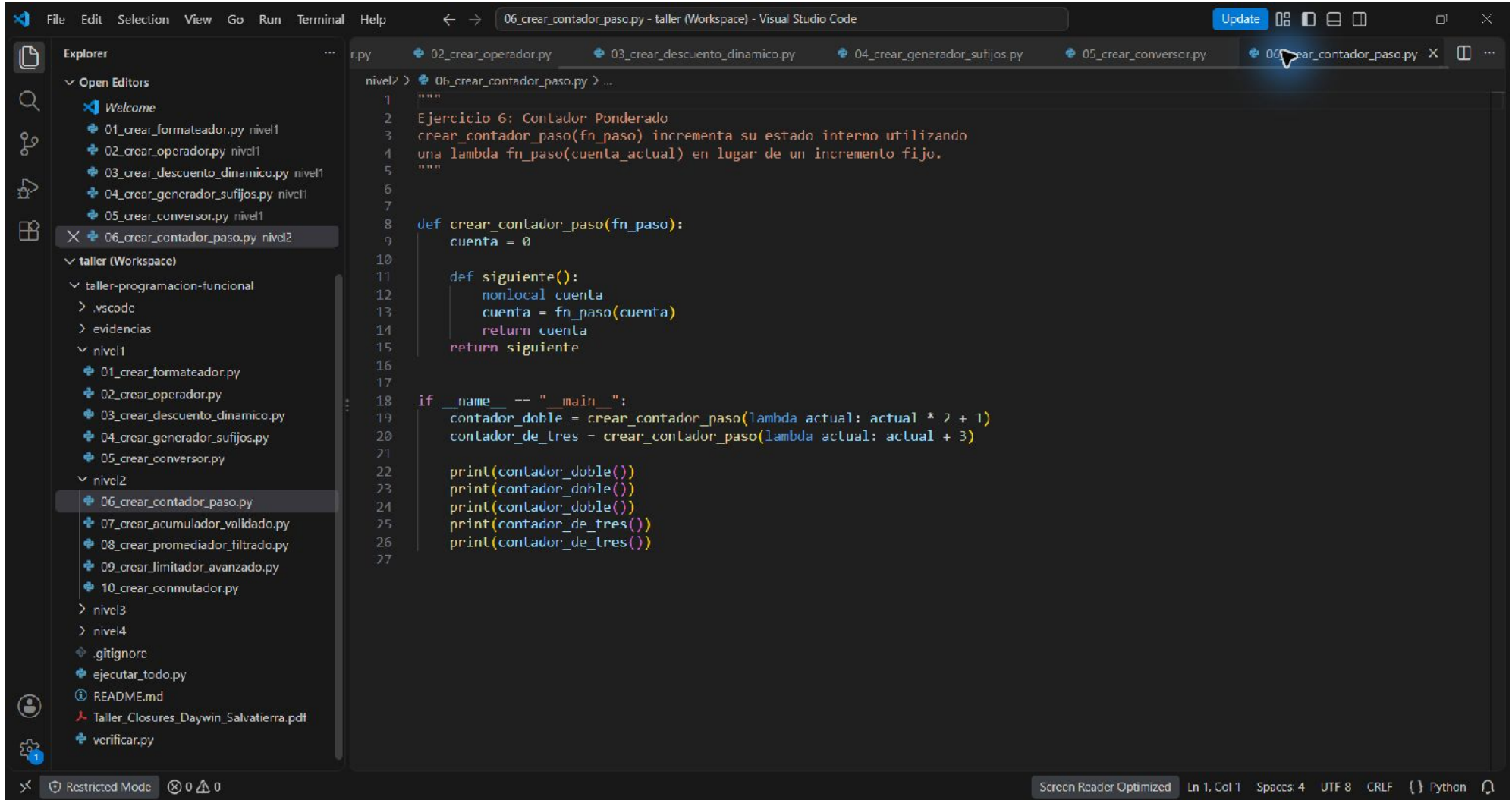
```
1 """
2 Ejercicio 5: Conversor de Divisas con Margen
3 crear_convertor(tasa, margen_lambda) retorna una funcion para convertir
4 montos calculando dinamicamente la comision adicional.
5 """
6
7
8 def crear_convertor(tasa, margen_lambda):
9     def convertir(monto):
10         convertido = monto * tasa
11         comision = margen_lambda(convertido)
12         return round(convertido + comision, 2)
13     return convertir
14
15
16 if __name__ == "__main__":
17     usd_a_eur = crear_convertor(0.92, lambda m: m * 0.02)
18     usd_a_pen = crear_convertor(3.75, lambda m: 5 if m > 200 else 1.5)
19
20     print(usd_a_eur(100))
21     print(usd_a_pen(50))
22     print(usd_a_pen(300))
23
```

Salida:

```
93.84
189.0
1130.0
```

6. Contador con actualización variable

La variable cuenta empieza en cero. La función interna usa nonlocal para actualizarla con el resultado de fn_paso. En esta solución, la lambda calcula el nuevo valor completo: una regla produce 1, 3 y 7; la otra avanza de tres en tres.



```
File Edit Selection View Go Run Terminal Help
06_crear_contador_paso.py - taller (Workspace) - Visual Studio Code
Update

Explorer
Open Editors
Welcome
01_crear_formateador.py nivel1
02_crear_operador.py nivel1
03_crear_descuento_dinamico.py nivel1
04_crear_generador_sufijos.py nivel1
05_crear_convertor.py nivel1
06_crear_contador_paso.py nivel2
taller (Workspace)
taller-programacion-funcional
.vscode
evidencias
nivel1
01_crear_formateador.py
02_crear_operador.py
03_crear_descuento_dinamico.py
04_crear_generador_sufijos.py
05_crear_convertor.py
nivel2
06_crear_contador_paso.py
07_crear_acumulador_validado.py
08_crear_promediador_filtrado.py
09_crear_limitador_avanzado.py
10_crear_conmutador.py
nivel3
nivel4
.gitignore
ejecutar_todo.py
README.md
Taller_Closures_Daywin_Salvatierra.pdf
verificar.py

06_crear_contador_paso.py
"""
Ejercicio 6: Contador Ponderado
crear_contador_paso(fn_paso) incrementa su estado interno utilizando
una lambda fn_paso(cuenta_actual) en lugar de un incremento fijo.
"""

def crear_contador_paso(fn_paso):
    cuenta = 0

    def siguiente():
        nonlocal cuenta
        cuenta = fn_paso(cuenta)
        return cuenta

    return siguiente

if __name__ == "__main__":
    contador_doble = crear_contador_paso(lambda actual: actual * 2 + 1)
    contador_de_tres = crear_contador_paso(lambda actual: actual + 3)

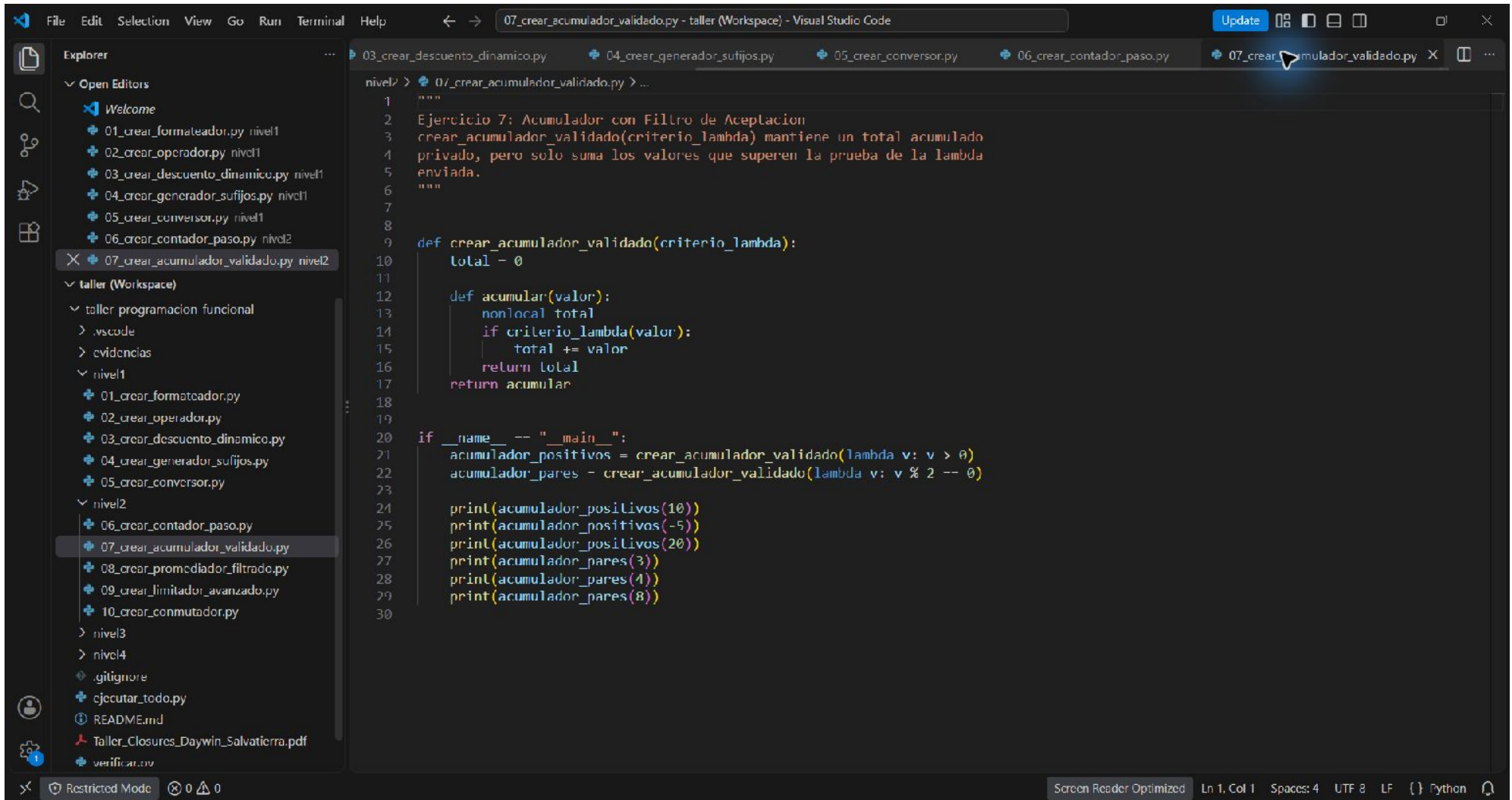
    print(contador_doble())
    print(contador_doble())
    print(contador_doble())
    print(contador_de_tres())
    print(contador_de_tres())
```

Salida:

```
1
3
7
3
6
```

7. Acumulador con filtro de aceptación

El acumulador solo añade un dato cuando la condición devuelve True. Una prueba acepta números positivos y otra acepta números pares. Un valor rechazado deja el total anterior intacto.



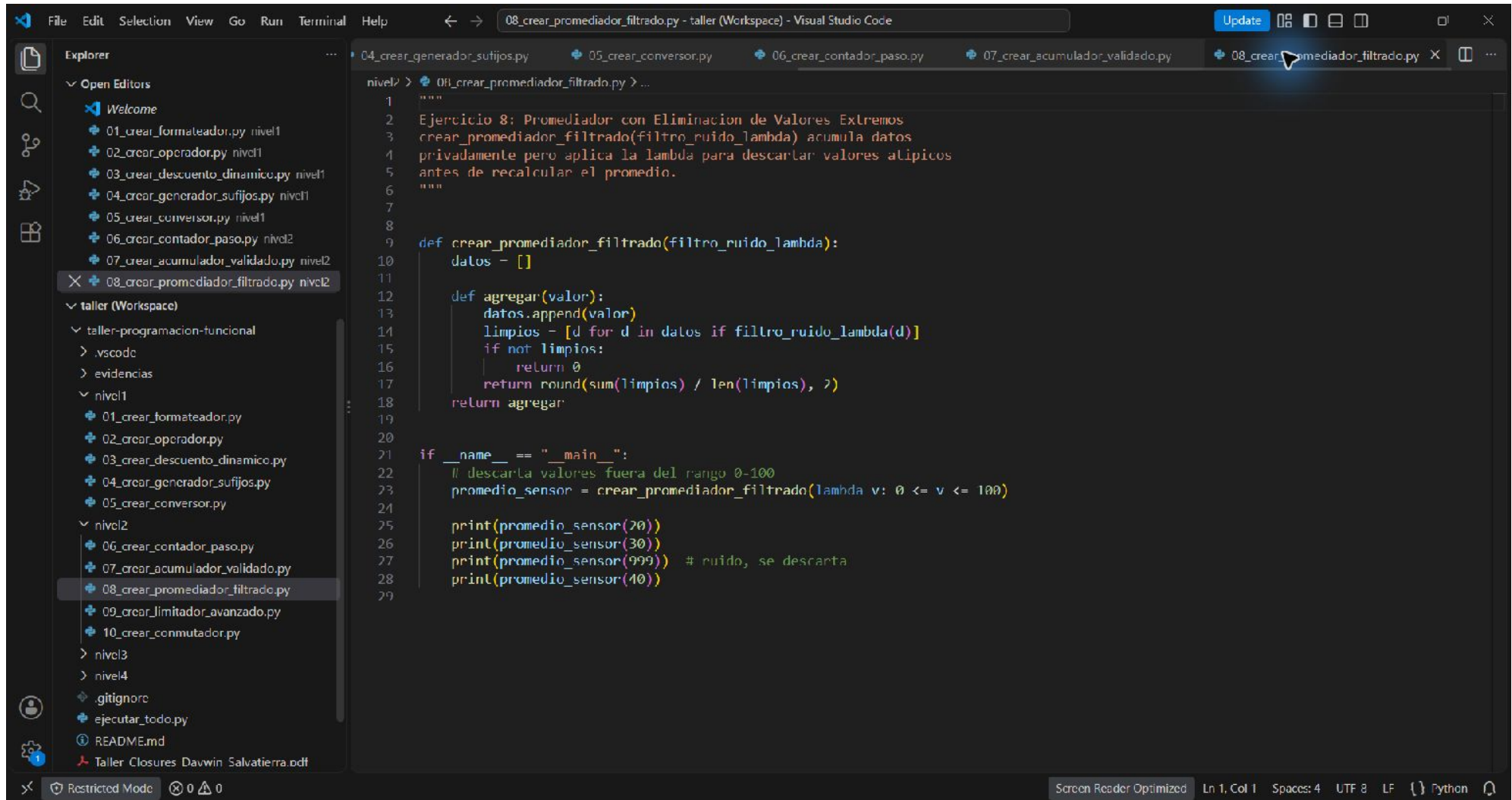
```
07_crear_acumulador_validado.py > ...
1 """
2 Ejercicio 7: Acumulador con Filtro de Aceptacion
3 crear_acumulador_validado(criterio_lambda) mantiene un total acumulado
4 privado, pero solo suma los valores que superen la prueba de la lambda
5 enviada.
6 """
7
8
9 def crear_acumulador_validado(criterio_lambda):
10     total = 0
11
12     def acumular(valor):
13         nonlocal total
14         if criterio_lambda(valor):
15             total += valor
16         return total
17     return acumular
18
19
20 if __name__ == "__main__":
21     acumulador_positivos = crear_acumulador_validado(lambda v: v > 0)
22     acumulador_pares = crear_acumulador_validado(lambda v: v % 2 == 0)
23
24     print(acumulador_positivos(10))
25     print(acumulador_positivos(-5))
26     print(acumulador_positivos(20))
27     print(acumulador_pares(3))
28     print(acumulador_pares(4))
29     print(acumulador_pares(8))
30
```

Salida:

```
10
10
30
0
4
12
```

8. Promedio sin valores fuera de rango

Los datos se guardan en una lista privada. Antes de calcular el promedio se conservan únicamente los que cumplen el filtro. La prueba acepta el rango de 0 a 100, por eso el valor 999 no altera el promedio.



The screenshot shows the Visual Studio Code editor with a workspace named 'taller (Workspace)'. The Explorer sidebar on the left shows a file tree with folders 'taller-programacion-funcional', 'evidencias', and 'nivel1'. The file '08_crear_promediador_filtrado.py' is selected under 'nivel2'. The main editor displays the code for this file. The code includes a docstring for 'Ejercicio 8: Promediador con Eliminación de Valores Extremos', a function 'crear_promediador_filtrado' that takes a filter function and returns a function 'agregar', and a main block that tests the function with values 20, 30, 999, and 40. The 'agregar' function filters out values not in the range 0-100 before calculating the average.

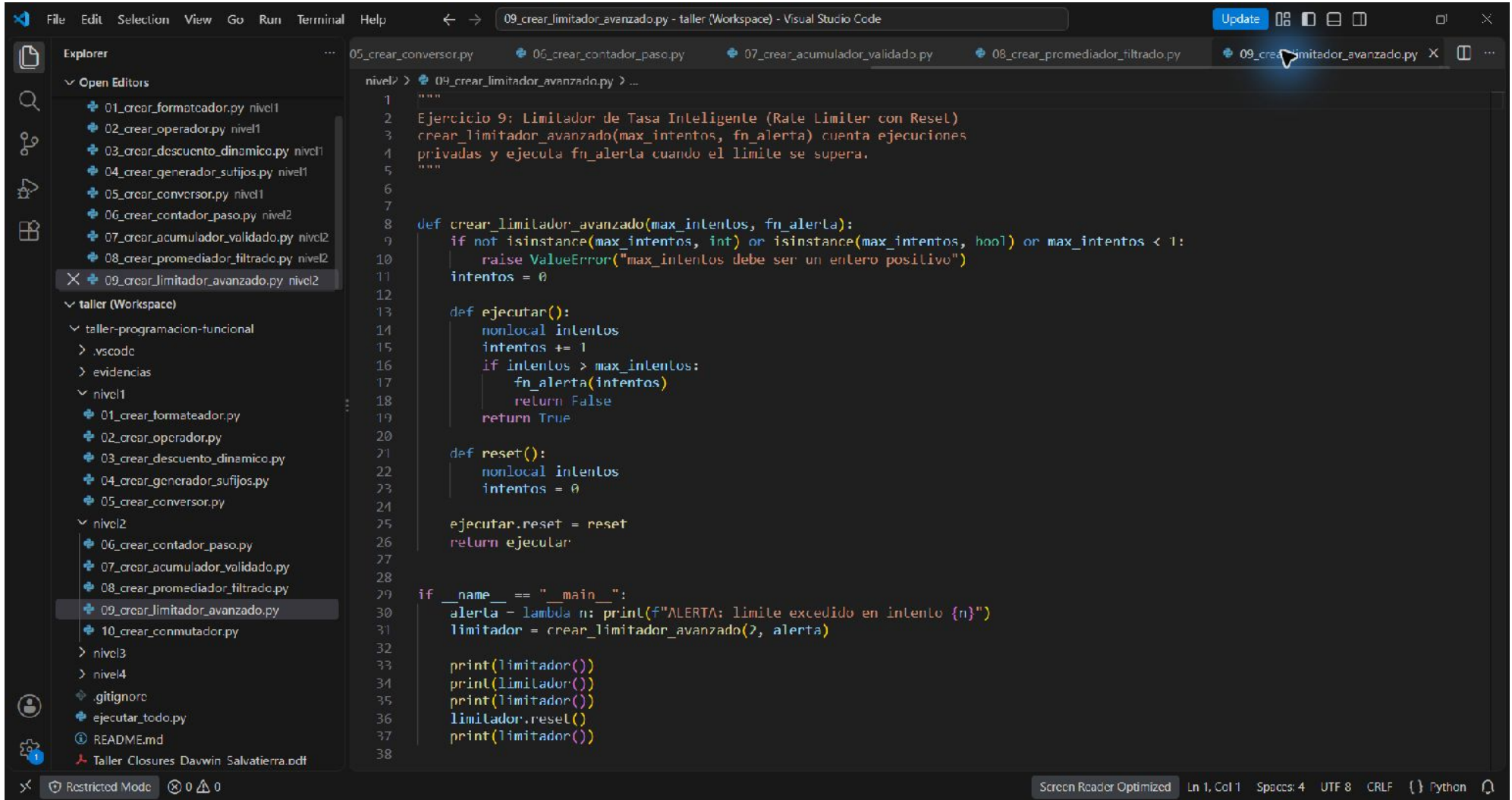
```
1 """
2 Ejercicio 8: Promediador con Eliminación de Valores Extremos
3 crear_promediador_filtrado(filtro_ruido_lambda) acumula datos
4 privadamente pero aplica la lambda para descartar valores atípicos
5 antes de recalcular el promedio.
6 """
7
8
9 def crear_promediador_filtrado(filtro_ruido_lambda):
10     datos = []
11
12     def agregar(valor):
13         datos.append(valor)
14         limpios = [d for d in datos if filtro_ruido_lambda(d)]
15         if not limpios:
16             return 0
17         return round(sum(limpios) / len(limpios), 2)
18     return agregar
19
20
21 if __name__ == "__main__":
22     # descarta valores fuera del rango 0-100
23     promedio_sensor = crear_promediador_filtrado(lambda v: 0 <= v <= 100)
24
25     print(promedio_sensor(20))
26     print(promedio_sensor(30))
27     print(promedio_sensor(999)) # ruido, se descarta
28     print(promedio_sensor(40))
29
```

Salida:

```
20.0
25.0
25.0
30.0
```

9. Limitador de intentos con reinicio

El closure cuenta las llamadas y permite hasta dos intentos en el ejemplo. Cuando se supera el máximo, ejecuta la alerta y devuelve False. El método reset restablece el contador para iniciar una nueva secuencia.



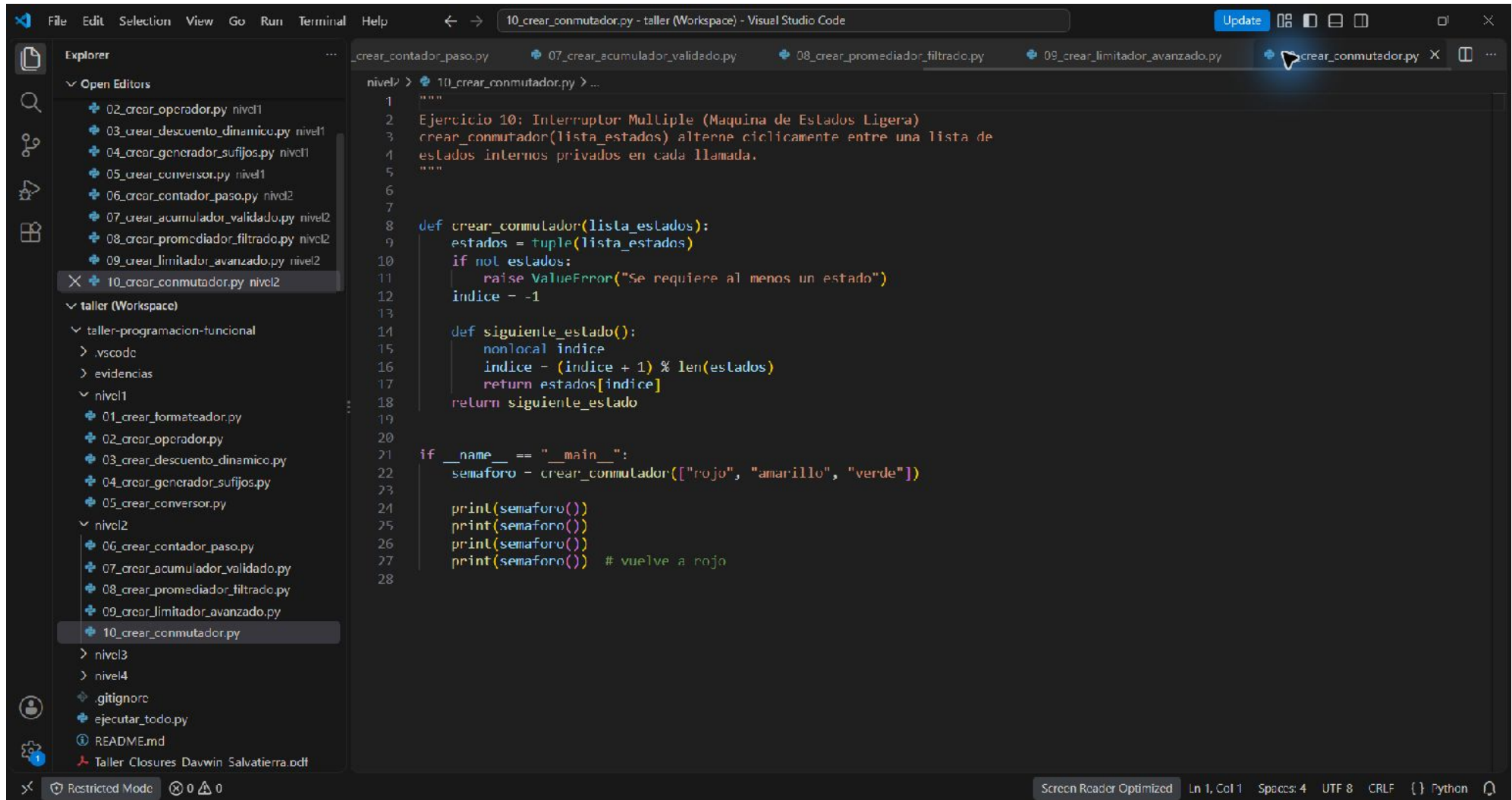
```
1 """
2 Ejercicio 9: Limitador de Tasa Inteligente (Rate Limiter con Reset)
3 crear_limitador_avanzado(max_intentos, fn_alerta) cuenta ejecuciones
4 privadas y ejecuta fn_alerta cuando el limite se supera.
5 """
6
7
8 def crear_limitador_avanzado(max_intentos, fn_alerta):
9     if not isinstance(max_intentos, int) or isinstance(max_intentos, bool) or max_intentos < 1:
10         raise ValueError("max_intentos debe ser un entero positivo")
11     intentos = 0
12
13     def ejecutar():
14         nonlocal intentos
15         intentos += 1
16         if intentos > max_intentos:
17             fn_alerta(intentos)
18             return False
19         return True
20
21     def reset():
22         nonlocal intentos
23         intentos = 0
24
25     ejecutar.reset = reset
26     return ejecutar
27
28
29 if __name__ == "__main__":
30     alerta = lambda n: print(f"ALERTA: limite excedido en intento {n}")
31     limitador = crear_limitador_avanzado(2, alerta)
32
33     print(limitador())
34     print(limitador())
35     print(limitador())
36     limitador.reset()
37     print(limitador())
38
```

Salida:

```
True
True
ALERTA: limite excedido en intento 3
False
True
```

10. Conmutador de estados

El índice avanza una posición en cada llamada. La operación módulo permite volver al principio cuando se alcanza el último estado. La secuencia se copia al crear el closure, de modo que modificar la lista original no altera el ciclo.



```
File Edit Selection View Go Run Terminal Help
10_crear_conmutador.py - taller (Workspace) - Visual Studio Code
Update

Explorer
Open Editors
02_crear_operador.py nivel1
03_crear_descuento_dinamico.py nivel1
04_crear_generador_sufijos.py nivel1
05_crear_convertor.py nivel1
06_crear_contador_paso.py nivel2
07_crear_acumulador_validado.py nivel2
08_crear_promediador_filtrado.py nivel2
09_crear_limitador_avanzado.py nivel2
X 10_crear_conmutador.py nivel2
taller (Workspace)
taller-programacion-funcional
> .vscode
> evidencias
> nivel1
01_crear_formateador.py
02_crear_operador.py
03_crear_descuento_dinamico.py
04_crear_generador_sufijos.py
05_crear_convertor.py
> nivel2
06_crear_contador_paso.py
07_crear_acumulador_validado.py
08_crear_promediador_filtrado.py
09_crear_limitador_avanzado.py
10_crear_conmutador.py
> nivel3
> nivel4
.gitignore
ejecutar_todo.py
README.md
Taller Closures Davwin Salvatierra.pdf

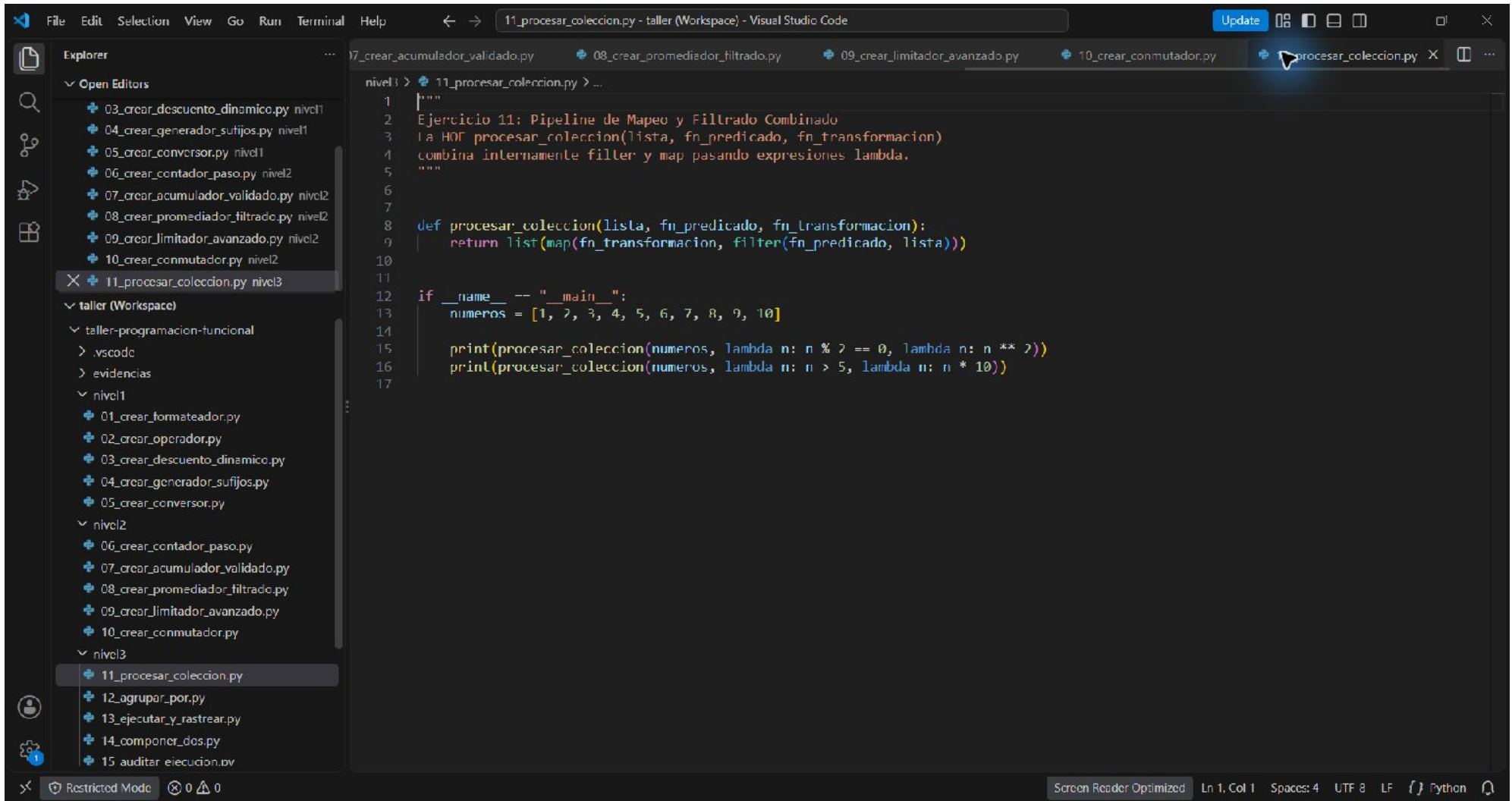
10_crear_conmutador.py
1 """
2 Ejercicio 10: Interruptor Multiple (Maquina de Estados Ligera)
3 crear_conmutador(lista_estados) alterne ciclicamente entre una lista de
4 estados internos privados en cada llamada.
5 """
6
7
8 def crear_conmutador(lista_estados):
9     estados = tuple(lista_estados)
10    if not estados:
11        raise ValueError("Se requiere al menos un estado")
12    indice = -1
13
14    def siguiente_estado():
15        nonlocal indice
16        indice = (indice + 1) % len(estados)
17        return estados[indice]
18    return siguiente_estado
19
20
21 if __name__ == "__main__":
22     semaforo = crear_conmutador(["rojo", "amarillo", "verde"])
23
24     print(semaforo())
25     print(semaforo())
26     print(semaforo())
27     print(semaforo()) # vuelve a rojo
28
```

Salida:

```
rojo
amarillo
verde
rojo
```

11. Filtrado y transformación de una colección

filter selecciona los valores que cumplen el predicado y map aplica la transformación a esos valores. La primera prueba obtiene cuadrados de números pares. La segunda multiplica por diez los números mayores de cinco.



```
11_procesar_coleccion.py - taller (Workspace) - Visual Studio Code

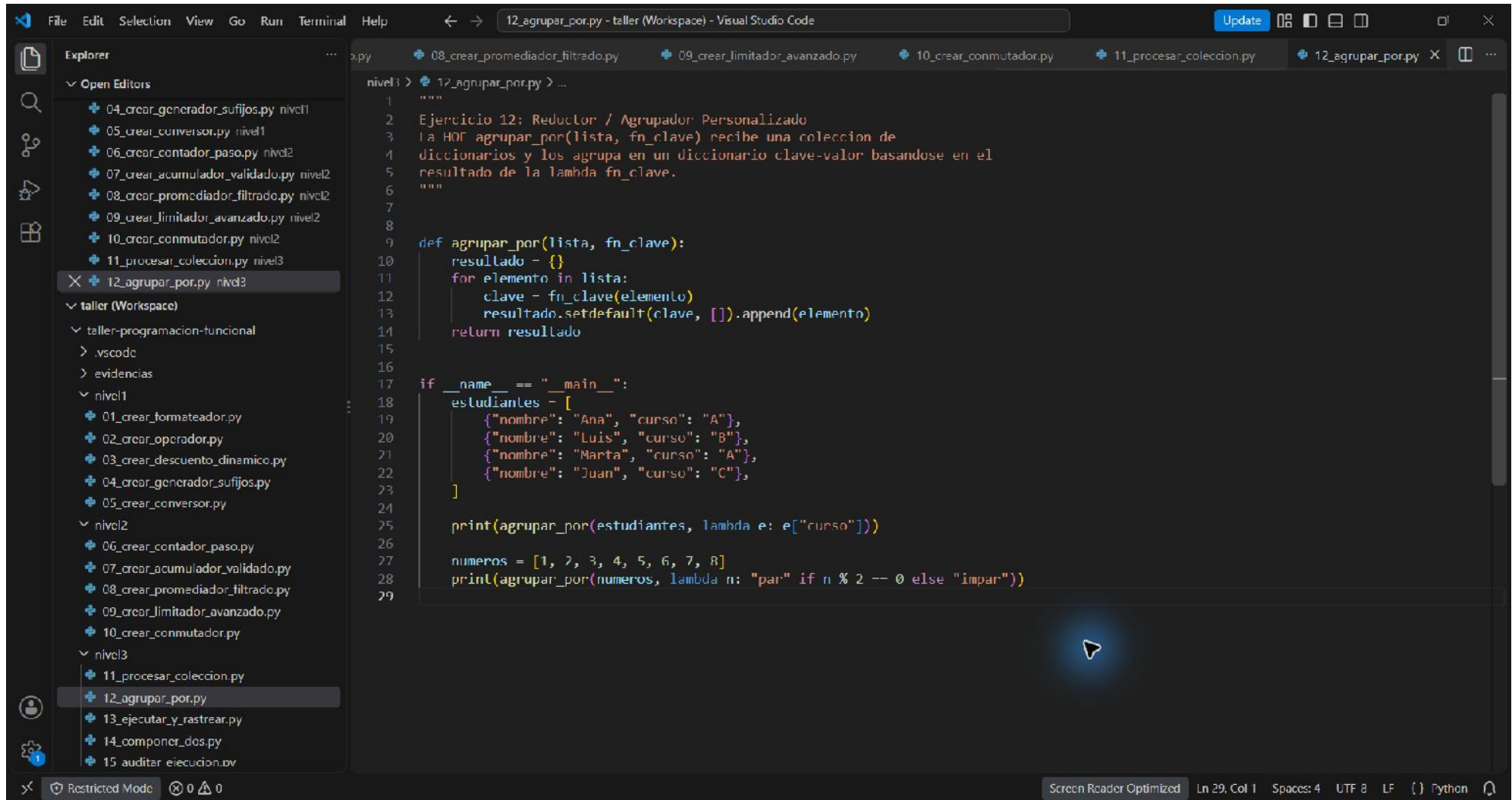
11_procesar_coleccion.py > ...
1  """
2  Ejercicio 11: Pipeline de Mapeo y Filtrado Combinado
3  La HOF procesar_coleccion(lista, fn_predicado, fn_transformacion)
4  combina internamente filter y map pasando expresiones lambda.
5  """
6
7
8  def procesar_coleccion(lista, fn_predicado, fn_transformacion):
9      return list(map(fn_transformacion, filter(fn_predicado, lista)))
10
11
12  if __name__ == "__main__":
13      numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
14
15      print(procesar_coleccion(numeros, lambda n: n % 2 == 0, lambda n: n ** 2))
16      print(procesar_coleccion(numeros, lambda n: n > 5, lambda n: n * 10))
17
```

Salida:

```
[4, 16, 36, 64, 100]
[60, 70, 80, 90, 100]
```

12. Agrupación mediante una clave

La lambda calcula la clave de cada elemento y el diccionario acumula los elementos de una misma clave en una lista. En el primer ejemplo se agrupan estudiantes por curso. En el segundo se separan números pares e impares.



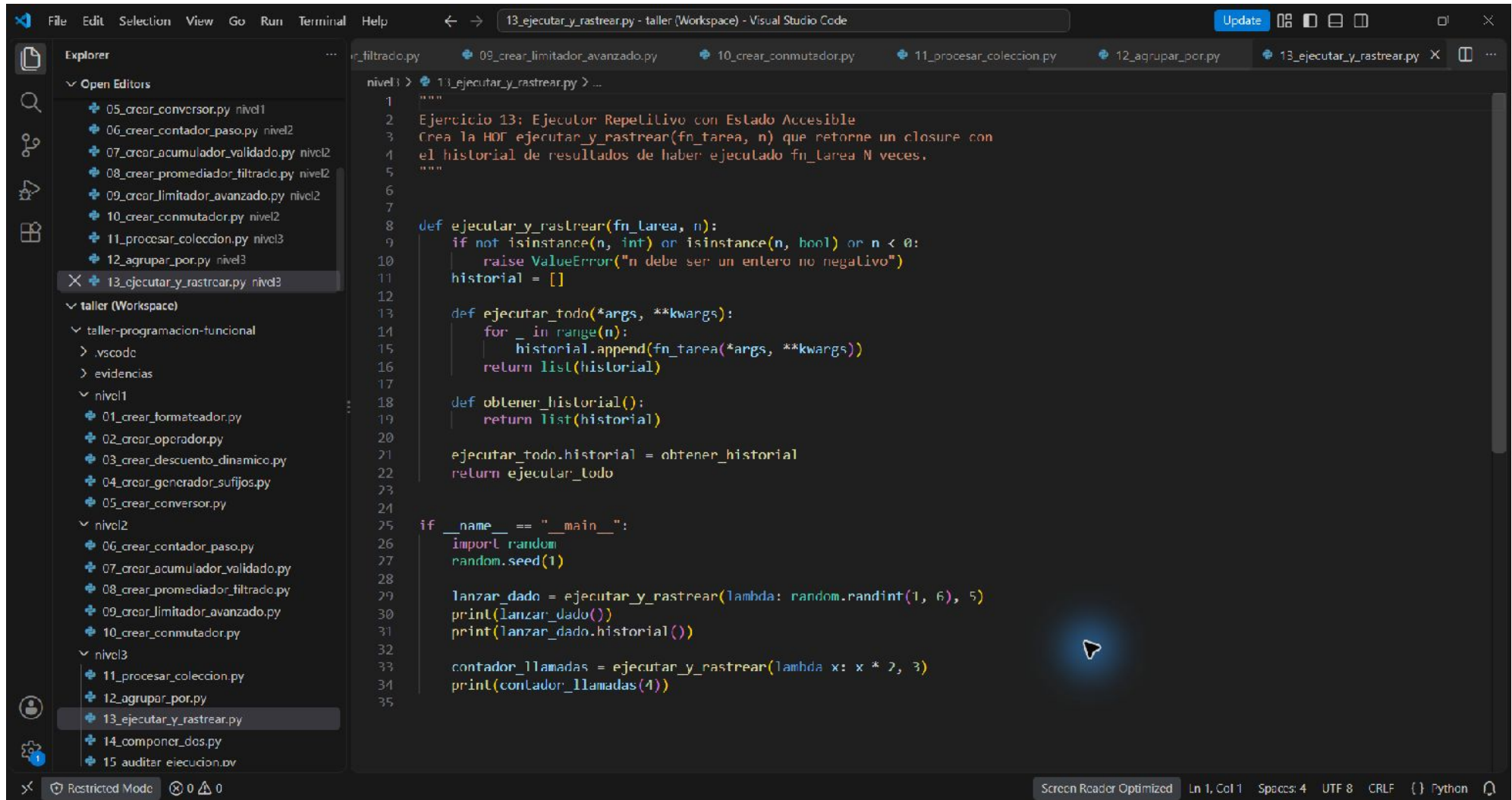
```
1 """
2 Ejercicio 12: Reductor / Agrupador Personalizado
3 La HOF agrupar_por(lista, fn_clave) recibe una coleccion de
4 diccionarios y los agrupa en un diccionario clave-valor basandose en el
5 resultado de la lambda fn_clave.
6 """
7
8
9 def agrupar_por(lista, fn_clave):
10     resultado = {}
11     for elemento in lista:
12         clave = fn_clave(elemento)
13         resultado.setdefault(clave, []).append(elemento)
14     return resultado
15
16
17 if __name__ == "__main__":
18     estudiantes = [
19         {"nombre": "Ana", "curso": "A"},
20         {"nombre": "Luis", "curso": "B"},
21         {"nombre": "Marta", "curso": "A"},
22         {"nombre": "Juan", "curso": "C"},
23     ]
24
25     print(agrupar_por(estudiantes, lambda e: e["curso"]))
26
27     numeros = [1, 2, 3, 4, 5, 6, 7, 8]
28     print(agrupar_por(numeros, lambda n: "par" if n % 2 == 0 else "impar"))
29
```

Salida:

```
{'A': [{'nombre': 'Ana', 'curso': 'A'}, {'nombre': 'Marta', 'curso': 'A'}], 'B': [{'nombre': 'Luis', 'curso': 'B'}], 'C': [{'nombre': 'Juan', 'curso': 'C'}]}
{'impar': [1, 3, 5, 7], 'par': [2, 4, 6, 8]}
```

13. Ejecución repetida e historial

Cada llamada al closure ejecuta la tarea N veces y añade los resultados a un historial privado. El método historial permite consultar lo acumulado sin volver a ejecutar la tarea. La prueba del dado usa una semilla para repetir sus resultados.



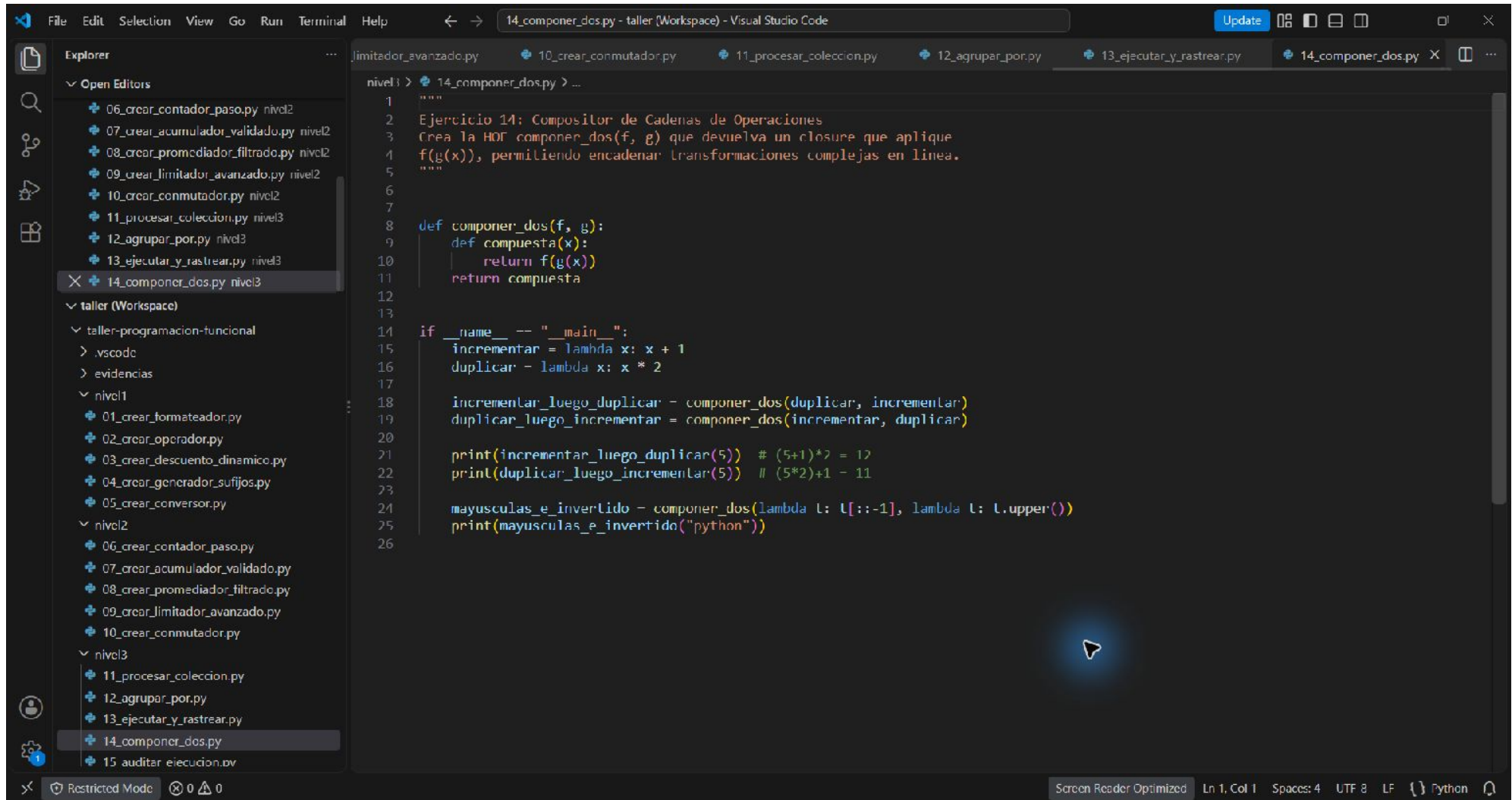
```
13_ejecutar_y_rastrear.py > ...
1 """
2 Ejercicio 13: Ejecutor Repetitivo con Estado Accesible
3 Crea la HOF ejecutar_y_rastrear(fn_tarea, n) que retorne un closure con
4 el historial de resultados de haber ejecutado fn_tarea N veces.
5 """
6
7
8 def ejecutar_y_rastrear(fn_tarea, n):
9     if not isinstance(n, int) or isinstance(n, bool) or n < 0:
10         raise ValueError("n debe ser un entero no negativo")
11     historial = []
12
13     def ejecutar_todo(*args, **kwargs):
14         for _ in range(n):
15             historial.append(fn_tarea(*args, **kwargs))
16         return list(historial)
17
18     def obtener_historial():
19         return list(historial)
20
21     ejecutar_todo.historial = obtener_historial
22     return ejecutar_todo
23
24
25 if __name__ == "__main__":
26     import random
27     random.seed(1)
28
29     lanzar_dado = ejecutar_y_rastrear(lambda: random.randint(1, 6), 5)
30     print(lanzar_dado())
31     print(lanzar_dado.historial())
32
33     contador_llamadas = ejecutar_y_rastrear(lambda x: x * 2, 3)
34     print(contador_llamadas(4))
35
```

Salida:

```
[2, 5, 1, 3, 1]
[2, 5, 1, 3, 1]
[8, 8, 8]
```

14. Composición de dos funciones

La función compuesta aplica primero g y después f. El orden importa: sumar uno y duplicar produce 12 para el valor 5, mientras que duplicar y sumar uno produce 11. La misma idea se aplica a las transformaciones de texto.



The screenshot shows the Visual Studio Code editor with a workspace named 'taller-programacion-funcional'. The file explorer on the left shows a directory structure with files organized by levels (nivel1, nivel2, nivel3). The file '14_componer_dos.py' is selected and open in the editor. The code in the editor defines a function 'componer_dos' that takes two functions 'f' and 'g' and returns a new function 'compuesta' that applies 'g' first and then 'f'. Below the function definition, there is a main block that demonstrates the function composition with two examples: one for numbers (incrementar and duplicar) and one for strings (invertir y mayusculas).

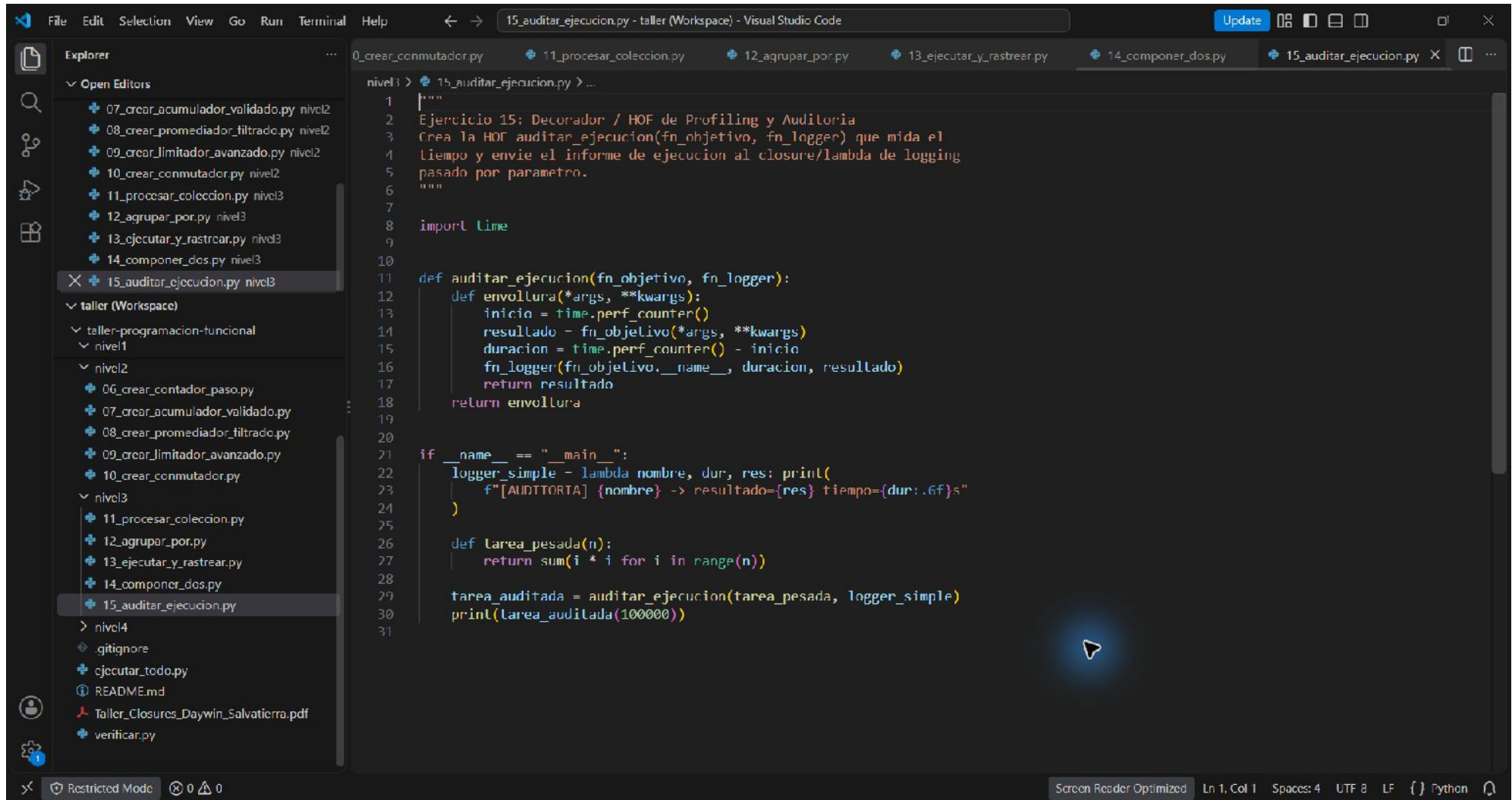
```
1 """
2 Ejercicio 14: Compositor de Cadenas de Operaciones
3 Crea la HOF componer_dos(f, g) que devuelva un closure que aplique
4 f(g(x)), permitiendo encadenar transformaciones complejas en linea.
5 """
6
7
8 def componer_dos(f, g):
9     def compuesta(x):
10         return f(g(x))
11     return compuesta
12
13
14 if __name__ == "__main__":
15     incrementar = lambda x: x + 1
16     duplicar = lambda x: x * 2
17
18     incrementar_luego_duplicar = componer_dos(duplicar, incrementar)
19     duplicar_luego_incrementar = componer_dos(incrementar, duplicar)
20
21     print(incrementar_luego_duplicar(5)) # (5+1)*2 = 12
22     print(duplicar_luego_incrementar(5)) # (5*2)+1 = 11
23
24     mayusculas_e_invertido = componer_dos(lambda t: t[::-1], lambda t: t.upper())
25     print(mayusculas_e_invertido("python"))
26
```

Salida:

```
12
11
NOHTYP
```

15. Tiempo de ejecución y auditoría

La envoltura toma una marca de tiempo antes y después de ejecutar la función. Luego envía al logger el nombre, la duración y el resultado. En la prueba se calcula la suma de cuadrados de los enteros entre 0 y 99999.



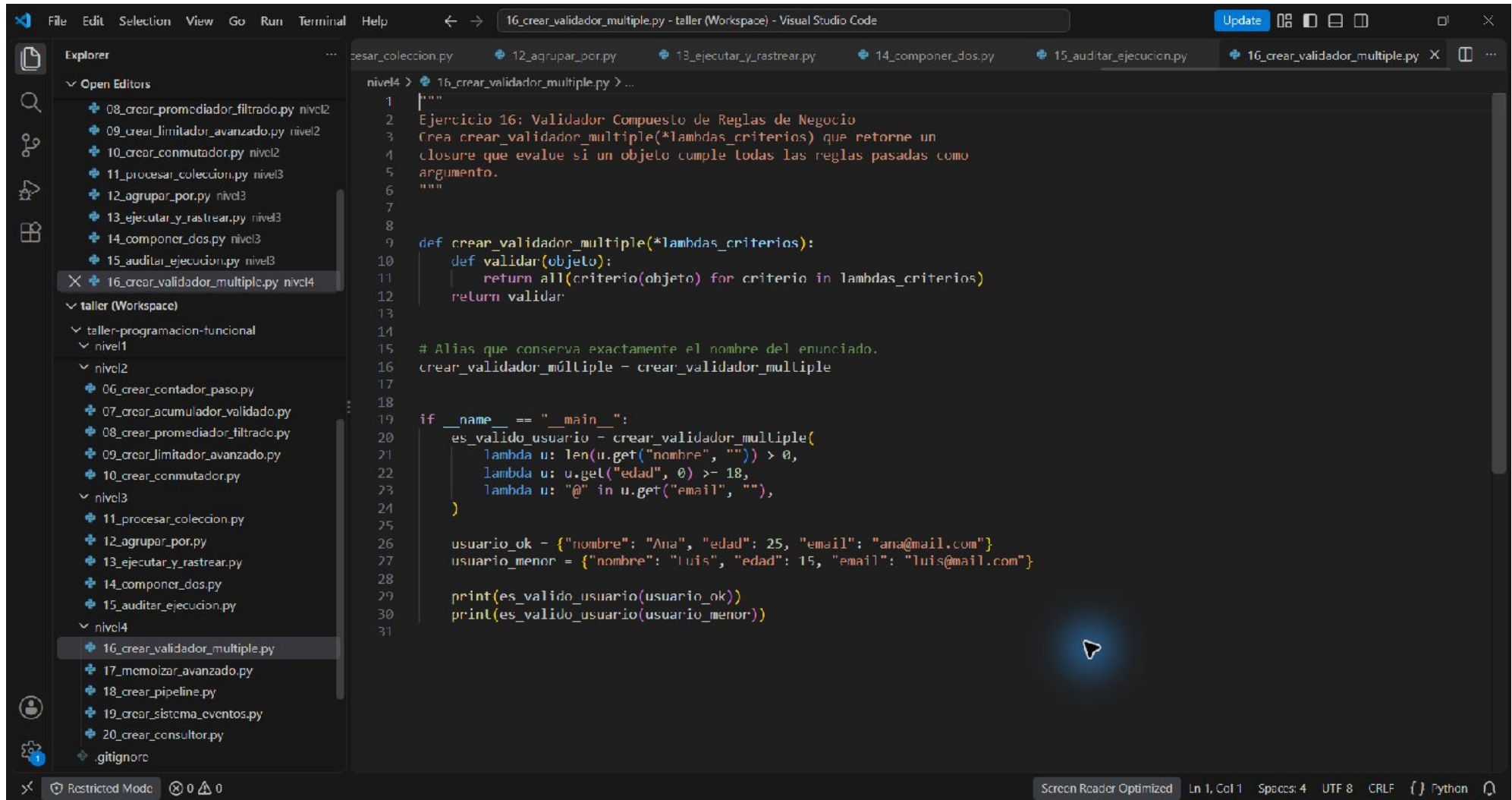
```
15_auditar_ejecucion.py > ...
1 """
2 Ejercicio 15: Decorador / HOF de Profiling y Auditoria
3 Crea la HOF auditar_ejecucion(fn_objetivo, fn_logger) que mida el
4 tiempo y envíe el informe de ejecución al closure/lambda de logging
5 pasado por parametro.
6 """
7
8 import time
9
10
11 def auditar_ejecucion(fn_objetivo, fn_logger):
12     def envoltura(*args, **kwargs):
13         inicio = time.perf_counter()
14         resultado = fn_objetivo(*args, **kwargs)
15         duracion = time.perf_counter() - inicio
16         fn_logger(fn_objetivo.__name__, duracion, resultado)
17         return resultado
18     return envoltura
19
20
21 if __name__ == "__main__":
22     logger_simple = lambda nombre, dur, res: print(
23         f"[AUDITORIA] {nombre} -> resultado={res} tiempo={dur:.6f}s"
24     )
25
26     def tarea_pesada(n):
27         return sum(i * i for i in range(n))
28
29     tarea_auditada = auditar_ejecucion(tarea_pesada, logger_simple)
30     print(tarea_auditada(100000))
31
```

Salida:

```
[AUDITORIA] tarea_pesada -> resultado=333328333350000 tiempo=0.014985s
333328333350000
```

16. Validación con varias reglas

all evalúa los criterios y acepta el objeto solo si todos se cumplen. El usuario de ejemplo necesita nombre, al menos 18 años y un correo que contenga @. El segundo usuario se rechaza porque tiene 15 años.



```
16_crear_validador_multiple.py - taller (Workspace) - Visual Studio Code

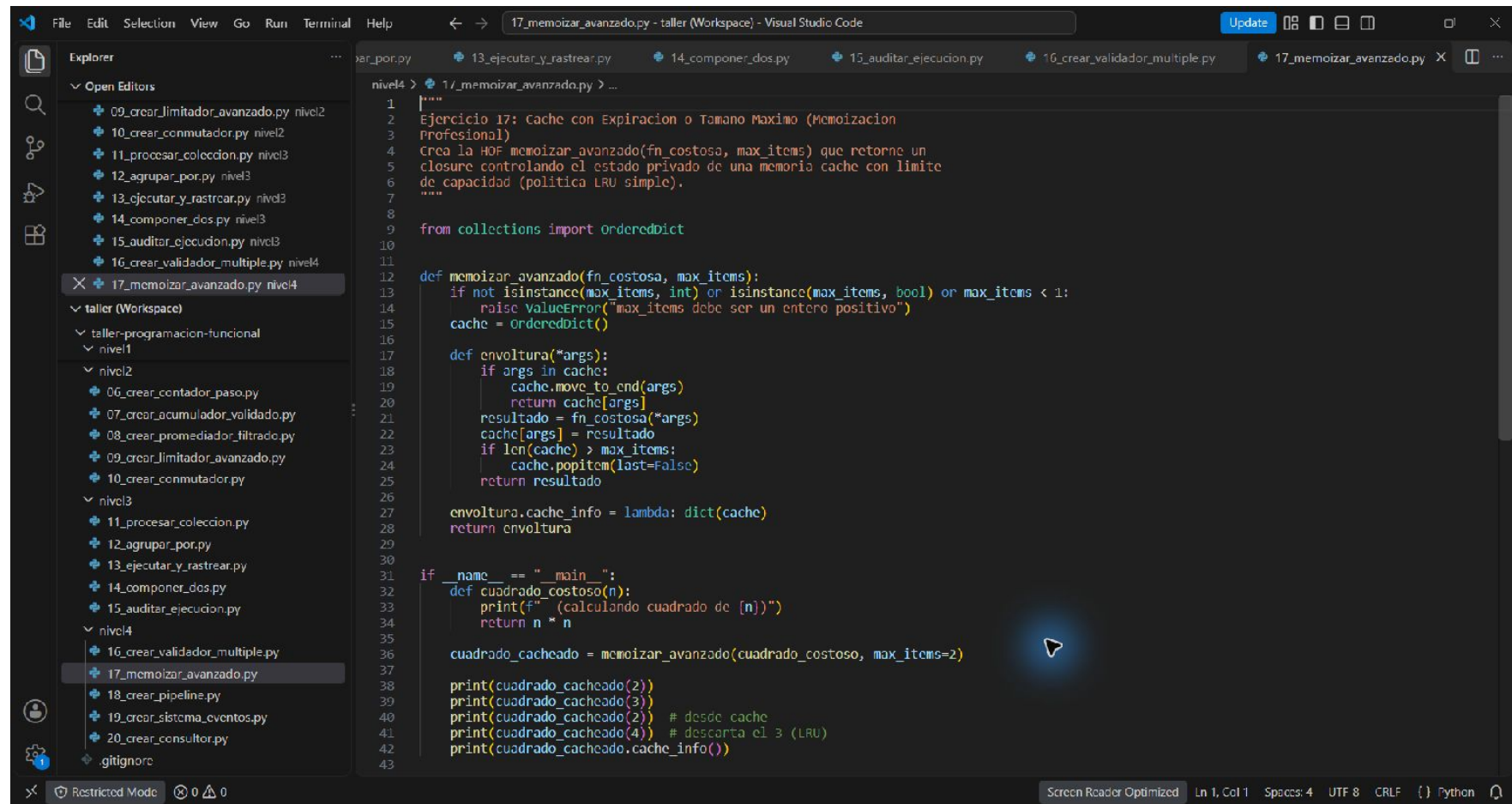
1 """
2 Ejercicio 16: Validador Compuesto de Reglas de Negocio
3 Crea crear_validador_multiple(*lambdas_criterios) que retorne un
4 Closure que evalúe si un objeto cumple todas las reglas pasadas como
5 argumento.
6 """
7
8
9 def crear_validador_multiple(*lambdas_criterios):
10     def validar(objeto):
11         return all(criterio(objeto) for criterio in lambdas_criterios)
12     return validar
13
14
15 # Alias que conserva exactamente el nombre del enunciado.
16 crear_validador_multiple = crear_validador_multiple
17
18
19 if __name__ == "__main__":
20     es_valido_usuario = crear_validador_multiple(
21         lambda u: len(u.get("nombre", "")) > 0,
22         lambda u: u.get("edad", 0) >= 18,
23         lambda u: "@" in u.get("email", ""),
24     )
25
26     usuario_ok = {"nombre": "Ana", "edad": 25, "email": "ana@mail.com"}
27     usuario_menor = {"nombre": "Luis", "edad": 15, "email": "luis@mail.com"}
28
29     print(es_valido_usuario(usuario_ok))
30     print(es_valido_usuario(usuario_menor))
31
```

Salida:

True
False

17. Memoización con capacidad limitada

La caché guarda resultados para reutilizarlos cuando se repiten los argumentos. Su política es LRU: se elimina la entrada que lleva más tiempo sin utilizarse. Consultar otra vez el 2 lo marca como reciente, de modo que al insertar el 4 se elimina el 3.



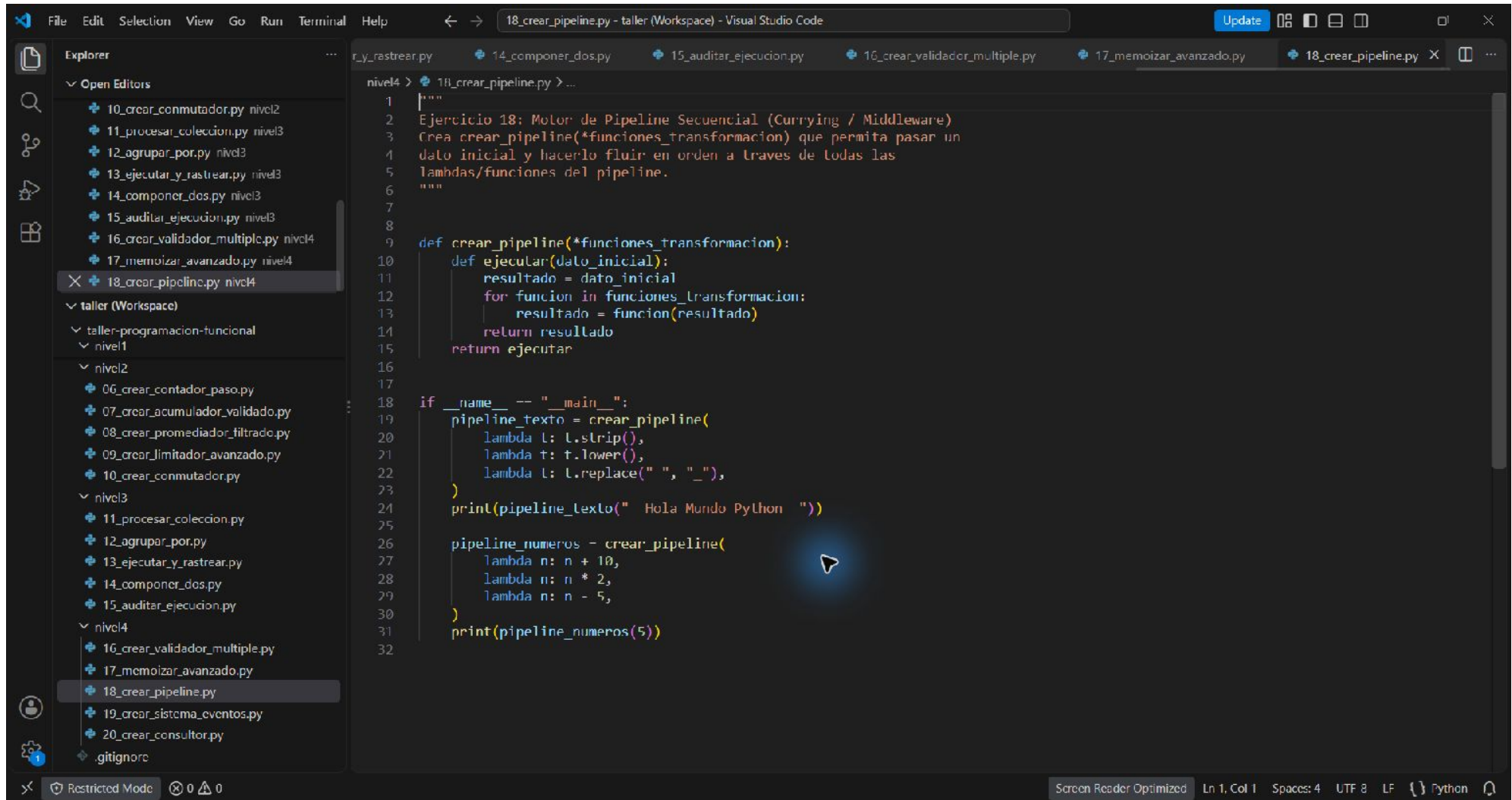
```
17_memoizar_avanzado.py > ...
1 """
2 Ejercicio 17: Cache con Expiracion o Tamano Maximo (Memoizacion
3 Profesional)
4 Crea la HOF memoizar_avanzado(fn costosa, max_items) que retorne un
5 closure controlando el estado privado de una memoria cache con limite
6 de capacidad (politica LRU simple).
7 """
8
9 from collections import OrderedDict
10
11
12 def memoizar_avanzado(fn_costosa, max_items):
13     if not isinstance(max_items, int) or isinstance(max_items, bool) or max_items < 1:
14         raise ValueError("max_items debe ser un entero positivo")
15     cache = OrderedDict()
16
17     def envoltura(*args):
18         if args in cache:
19             cache.move_to_end(args)
20             return cache[args]
21         resultado = fn_costosa(*args)
22         cache[args] = resultado
23         if len(cache) > max_items:
24             cache.popitem(last=False)
25         return resultado
26
27     envoltura.cache_info = lambda: dict(cache)
28     return envoltura
29
30
31 if __name__ == "__main__":
32     def cuadrado_costoso(n):
33         print(f" (calculando cuadrado de {n})")
34         return n * n
35
36     cuadrado_cacheado = memoizar_avanzado(cuadrado_costoso, max_items=2)
37
38     print(cuadrado_cacheado(2))
39     print(cuadrado_cacheado(3))
40     print(cuadrado_cacheado(2)) # desde cache
41     print(cuadrado_cacheado(4)) # descarta el 3 (LRU)
42     print(cuadrado_cacheado.cache_info())
43
```

Salida:

```
(calculando cuadrado de 2)
4
(calculando cuadrado de 3)
9
4
(calculando cuadrado de 4)
16
{(2,): 4, (4,): 16}
```

18. Pipeline de transformaciones

El dato inicial pasa por las funciones en el orden en que se declararon. El ejemplo de texto elimina espacios exteriores, convierte a minúsculas y sustituye espacios por guiones bajos. El ejemplo numérico suma, multiplica y resta.



The screenshot shows the Visual Studio Code editor with the file `18_crear_pipeline.py` open. The Explorer sidebar on the left shows a project structure with folders for different levels of a course. The main editor displays the following Python code:

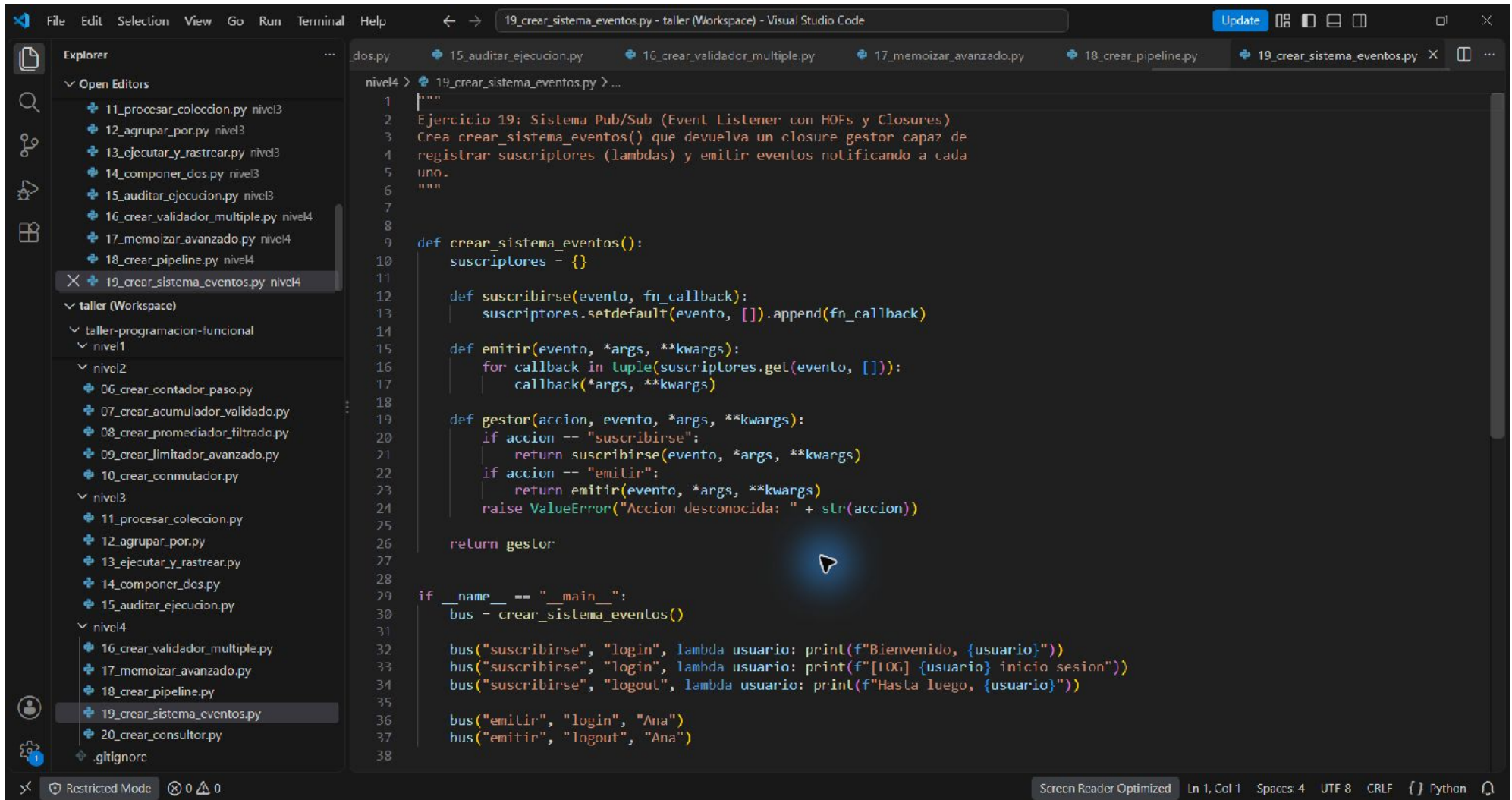
```
nivel4 > 18_crear_pipeline.py > ...
1  """
2  Ejercicio 18: Motor de Pipeline Secuencial (Currying / Middleware)
3  Crea crear_pipeline(*funciones_transformacion) que permita pasar un
4  dato inicial y hacerlo fluir en orden a traves de todas las
5  lambdas/funciones del pipeline.
6  """
7
8
9  def crear_pipeline(*funciones_transformacion):
10     def ejecutar(dato_inicial):
11         resultado = dato_inicial
12         for funcion in funciones_transformacion:
13             resultado = funcion(resultado)
14         return resultado
15     return ejecutar
16
17
18 if __name__ == "__main__":
19     pipeline_texto = crear_pipeline(
20         lambda t: t.strip(),
21         lambda t: t.lower(),
22         lambda t: t.replace(" ", "_"),
23     )
24     print(pipeline_texto("  Hola Mundo Python  "))
25
26     pipeline_numeros = crear_pipeline(
27         lambda n: n + 10,
28         lambda n: n * 2,
29         lambda n: n - 5,
30     )
31     print(pipeline_numeros(5))
32
```

Salida:

```
hola_mundo_python
25
```

19. Publicación y suscripción de eventos

El gestor es un closure que conserva los suscriptores de cada evento. La acción suscribirse registra una función; emitir llama a las funciones asociadas. En la prueba, login notifica a dos suscriptores y logout a uno.



The screenshot shows the Visual Studio Code editor with a workspace named 'taller (Workspace)'. The Explorer sidebar on the left lists various Python files organized by levels (nivel1, nivel2, nivel3, nivel4). The file '19_crear_sistema_eventos.py' is selected and open in the editor. The code in the editor implements an event system with the following components:

- A docstring for 'Ejercicio 19: Sistema Pub/Sub (Event Listener con HOFs y Closures)'.
- A function `crear_sistema_eventos()` that returns a `gestor` closure. It initializes a dictionary `suscriptores`.
- A function `suscribirse(evento, fn_callback)` that adds a callback to the list of subscribers for a specific event.
- A function `emitir(evento, *args, **kwargs)` that iterates over the subscribers for a given event and calls their callback functions with the provided arguments.
- A closure `gestor` that handles three actions: 'suscribirse' (calls `suscribirse`), 'emitir' (calls `emitir`), and raises a `ValueError` for unknown actions.
- A main block that creates the `bus` object and registers callbacks for 'login' and 'logout' events.

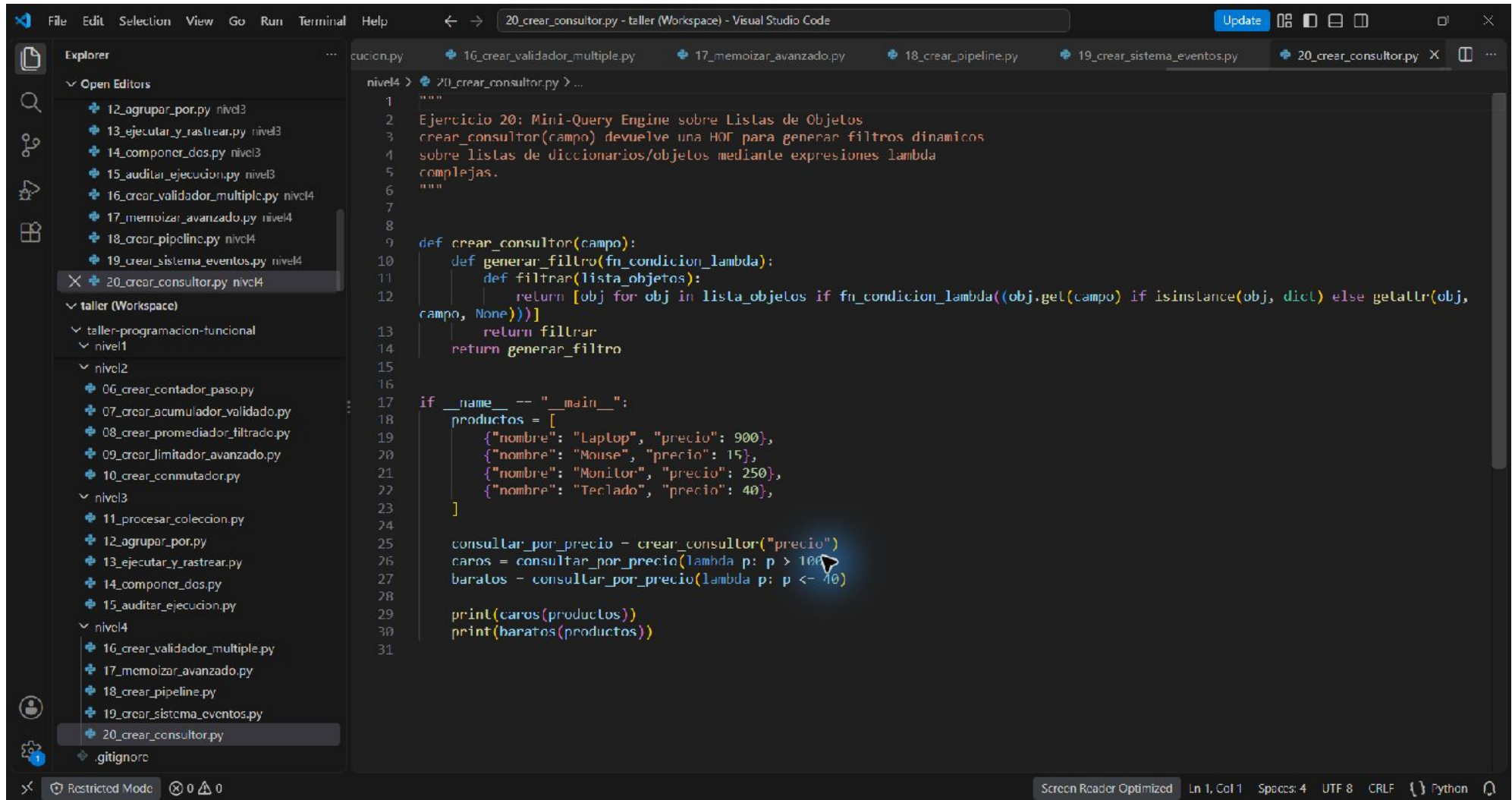
```
1 """
2 Ejercicio 19: Sistema Pub/Sub (Event Listener con HOFs y Closures)
3 Crea crear_sistema_eventos() que devuelva un closure gestor capaz de
4 registrar suscriptores (lambdas) y emitir eventos notificando a cada
5 uno.
6 """
7
8
9 def crear_sistema_eventos():
10     suscriptores = {}
11
12     def suscribirse(evento, fn_callback):
13         suscriptores.setdefault(evento, []).append(fn_callback)
14
15     def emitir(evento, *args, **kwargs):
16         for callback in tuple(suscriptores.get(evento, [])):
17             callback(*args, **kwargs)
18
19     def gestor(accion, evento, *args, **kwargs):
20         if accion == "suscribirse":
21             return suscribirse(evento, *args, **kwargs)
22         if accion == "emitir":
23             return emitir(evento, *args, **kwargs)
24         raise ValueError("Accion desconocida: " + str(accion))
25
26     return gestor
27
28
29 if __name__ == "__main__":
30     bus = crear_sistema_eventos()
31
32     bus("suscribirse", "login", lambda usuario: print(f"Bienvenido, {usuario}"))
33     bus("suscribirse", "login", lambda usuario: print(f"[LOG] {usuario} inicio sesion"))
34     bus("suscribirse", "logout", lambda usuario: print(f"Hasta luego, {usuario}"))
35
36     bus("emitir", "login", "Ana")
37     bus("emitir", "logout", "Ana")
38
```

Salida:

```
Bienvenido, Ana
[LOG] Ana inicio sesion
Hasta luego, Ana
```

20. Consultas dinámicas sobre objetos

El primer closure conserva el campo que se consultará. Después recibe una condición y devuelve el filtro que procesa la lista. Esto permite crear consultas de productos caros y baratos usando el mismo campo precio.



The screenshot shows the Visual Studio Code editor with a Python file named `20_crear_consultor.py` open. The file is part of a workspace named 'taller (Workspace)'. The Explorer sidebar on the left shows a directory structure with files organized by levels (nivel1, nivel2, nivel3, nivel4). The main editor window displays the following Python code:

```
1 """
2 Ejercicio 20: Mini-Query Engine sobre Listas de Objetos
3 crear_consultor(campo) devuelve una HOF para generar filtros dinamicos
4 sobre listas de diccionarios/objetos mediante expresiones lambda
5 complejas.
6 """
7
8
9 def crear_consultor(campo):
10     def generar_filtro(fn_condicion_lambda):
11         def filtrar(lista_objetos):
12             return [obj for obj in lista_objetos if fn_condicion_lambda((obj.get(campo) if isinstance(obj, dict) else getattr(obj,
13 campo, None)))]
14         return filtrar
15     return generar_filtro
16
17 if __name__ == "__main__":
18     productos = [
19         {"nombre": "Laptop", "precio": 900},
20         {"nombre": "Mouse", "precio": 15},
21         {"nombre": "Monitor", "precio": 250},
22         {"nombre": "Teclado", "precio": 40},
23     ]
24
25     consultar_por_precio = crear_consultor("precio")
26     caros = consultar_por_precio(lambda p: p > 100)
27     baratos = consultar_por_precio(lambda p: p <= 40)
28
29     print(caros(productos))
30     print(baratos(productos))
31
```

Salida:

```
[{'nombre': 'Laptop', 'precio': 900}, {'nombre': 'Monitor', 'precio': 250}]
[{'nombre': 'Mouse', 'precio': 15}, {'nombre': 'Teclado', 'precio': 40}]
```